TaylorFlow-Auswertung

Handbuch

29.10.2026 · Stand 8e0a1ea

Inhaltsverzeichnis

1 Installation **4**
- 1.1 Python installieren . . . 4
- 1.2 Programm herunterladen . . . 4
- 1.3 Messdaten bereitstellen . . . 5
- 1.4 Einrichtung . . . 5
- 1.5 Erster Start . . . 6
- 1.6 Arbeiten in VS Code oder Spyder . . . 6
- 1.7 Typische Fehler . . . 6

2 Messdaten **8**
- 2.1 Herkunft . . . 8
- 2.2 Dateinamen . . . 8
- 2.3 Dateiinhalt . . . 9
- 2.4 Ausschlussregeln . . . 9
- 2.5 Automatischer Kamerafilter . . . 10
- 2.6 Neue Messreihe auswerten . . . 10

3 Erste Auswertung **12**
- 3.1 Programmteile im Überblick . . . 12
- 3.2 Starten . . . 12
- 3.3 Messdateien auswählen . . . 12
- 3.4 Ergebnisse finden und prüfen . . . 13
- 3.5 Häufige Varianten . . . 13

4 Plots auswählen und anpassen **15**
- 4.1 Grundprinzip . . . 15
- 4.2 Blasenlängen-Fit (`blasenlaengen_fit`) . . . 17
- 4.3 Absorbierte Stoffmenge (`stoffmenge`) . . . 18
- 4.4 Druckprofil (`druckprofil`) . . . 19
- 4.5 kLa-Fläche (`kla_flaeche`) . . . 20
- 4.6 Volumenspezifischer Molenstrom (`nspez_flaeche`) . . . 21
- 4.7 Modellflächen (`modell_flaechen`) . . . 22
- 4.8 Beliebige Spalten als Fläche (`spalten_flaechen`) . . . 22
- 4.9 Kennzahlen-Übersicht (`heatmaps`) . . . 24
- 4.10 Darstellung aller Plots (`STIL`) . . . 25
- 4.11 Eine einzelne Abbildung erzeugen (`abbildung.py`) . . . 26
- 4.12 Rezepte . . . 26
- 4.13 Plots direkt aus Python . . . 27

5 Konfiguration **28**
- 5.1 Ordner und Dateien . . . 28
- 5.2 Umfang der Auswertung . . . 28
- 5.3 Auswerte-Parameter (`PARAMETER`) . . . 29
- 5.4 Weitere Abschnitte . . . 30

6 Ergebnisse **32**
- 6.1 Ablauf der Auswertung . . . 32
- 6.2 Ergebnisdateien . . . 32
- 6.3 Konsolenausgabe . . . 34
- 6.4 Warnungsprotokoll . . . 34

7 Methodik **36**
- 7.1 Annahmen . . . 36

7.2 Blasenlängen-Fit . . . 36
7.3 Filmdicke und Blasengeometrie . . . 37
7.4 Druckprofil . . . 37
7.5 Stoffbilanz . . . 38
7.6 Aktive Zone und kLa_log_mean . . . 38
7.7 Weitere Kenngrößen . . . 39
7.8 Messunsicherheit . . . 39
7.9 Literatur . . . 39

8 Grenzen **41**
8.1 Gültigkeitsbereiche der Modelle . . . 41
8.2 Bekannte Einschränkungen . . . 41
8.3 Empfindlichkeit gegenüber den Einstellungen . . . 42
8.4 Offene Punkte . . . 43

9 Literaturvergleich und eigene Modelle **44**
9.1 Ablauf . . . 44
9.2 Literaturmodelle . . . 45
9.3 Eigenes Modell anpassen . . . 46
9.4 Aus Python . . . 48
9.5 Neues Literaturmodell als Formel ergänzen . . . 48

10 Druckkalibrierung **49**
10.1 Wozu . . . 49
10.2 Verfahren . . . 49
10.3 Nutzung in der Auswertung . . . 50
10.4 Neu kalibrieren . . . 50
10.5 Offene Fragen . . . 51

11 Monolith **52**
11.1 Messdaten und Einstellungen . . . 52
11.2 Rechnung . . . 52
11.3 Ausgaben . . . 53
11.4 Einordnung . . . 55

12 Programmaufbau und Erweiterung **56**
12.1 Aufbau . . . 56
12.2 Tests . . . 56
12.3 Typische Erweiterungen . . . 57
12.4 Handbuch erzeugen . . . 58

1 Installation

1.1 Python installieren

Benötigt wird **Python 3.10 oder neuer** (getestet mit Python 3.12) mit diesen Paketen. Sie werden in Abschnitt 1.4 mit einem einzigen Befehl installiert:

Paket	Wofür
numpy, pandas, scipy	Rechnung, Tabellen, Kurvenanpassung
matplotlib	alle Plots
thermo	Stoffdaten von Wasser (Dichte, Viskosität, Oberflächenspannung, Dampfdruck)
pyyaml	Ausschlussregeln und eigene Modelle (`.yaml`-Dateien)
pytest	Prüfung der Installation

Das Auswahlfenster für die Messdateien nutzt `tkinter`. Es ist Teil der Python-Installation von python.org und muss nicht eigens installiert werden.

Python neu installieren (Windows)

1. Auf https://www.python.org/downloads/ den Installer der aktuellen Version 3.12 oder 3.13 herunterladen (*Windows installer (64-bit)*).
2. Den Installer starten. Im ersten Fenster **unten den Haken bei „Add python.exe to PATH“ setzen**, dann **Install Now** wählen.
3. Die Voreinstellungen enthalten bereits *tcl/tk and IDLE*; das wird für das Auswahlfenster gebraucht. Bei **Customize installation** diesen Haken nicht entfernen.
4. Installation abschließen.

Prüfen

Die Eingabeaufforderung öffnen (Startmenü → „cmd“) und eingeben:

```
python --version
```

Erwartet wird z. B. `Python 3.12.7`. Erscheint stattdessen der Microsoft Store oder „Python wurde nicht gefunden“, fehlt der Haken bei „Add to PATH“: den Installer erneut starten, **Modify → Next →** *Add Python to environment variables* anhaken.

1.2 Programm herunterladen

Das Programm liegt auf GitHub: https://github.com/Oxymoron9946/TaylorFlow_Auswertung (Branch `main`). Das Repository ist öffentlich; zum Herunterladen ist kein GitHub-Konto nötig.

Variante A: als ZIP, ohne weitere Software

1. Die Seite im Browser öffnen.
2. Oben rechts auf **Code → Download ZIP** klicken.

3. Das ZIP entpacken, z. B. nach `C:\Messungen\TaylorFlow_Auswertung`. Dieser Ordner ist im Folgenden der **Projektordner**; er enthält u. a. `config.py`, `auswertung.py` und `abbildung.py`.

Für eine neuere Version das ZIP erneut herunterladen und den Ordner ersetzen. Eigene Änderungen an `config.py`, `config/ausschluesse.yaml` und `config/eigene_modelle.yaml` vorher sichern.

Variante B: mit Git (bequem für spätere Aktualisierungen)

1. *Git for Windows* von https://git-scm.com/download/win installieren (Standardeinstellungen genügen).
2. Die Eingabeaufforderung öffnen, in den gewünschten Elternordner wechseln und das Repository holen:

   ```
   cd C:\Messungen
   git clone https://github.com/Oxymoron9946/TaylorFlow_Auswertung.git
   ```

3. Später eine neue Version holen (im Projektordner):

   ```
   cd C:\Messungen\TaylorFlow_Auswertung
   git pull
   ```

   Hat man `config.py` lokal geändert, meldet `git pull` einen Konflikt nur dann, wenn dieselbe Datei auch auf GitHub geändert wurde.

1.3 Messdaten bereitstellen

Die Auswertung liest die Ergebnisdateien der **Videoauswertung** (Handbuch der Videoauswertung, Kapitel 4.2):

Datei der Videoauswertung	Wofür hier
`output/Output_MassTransfer/Auswertung_Masse_<…>.csv`	Stoffübergang, `auswertung.py`
`output/Output_PressureCalibration/Auswertung_Druck_<…>.csv`	Druckkalibrierung, `kalibrierung.py`

Die Dateien müssen nicht kopiert werden: Beim Start öffnet sich ein Auswahlfenster, in dem sie direkt im Ausgabeordner der Videoauswertung gewählt werden. Damit das Fenster gleich dort startet, in `config.py` den Ordner eintragen:

```
DIALOG_ORDNER = Path(r"C:\Messungen\Videoauswertung\output\Output_MassTransfer")
```

Die Messdaten des Forschungspraktikums liegen bereits im Projektordner unter `data/raw/`. Mit ihnen lässt sich alles ausprobieren. Aufbau der Dateien: Kapitel 2.

1.4 Einrichtung

Im Projektordner wird eine eigene Python-Umgebung (*virtual environment*, Ordner `.venv`) angelegt. Sie enthält genau die benötigten Pakete und stört keine andere Python-Installation.

1. Eingabeaufforderung öffnen und in den Projektordner wechseln:

   ```
   cd C:\Messungen\TaylorFlow_Auswertung
   ```

2. Umgebung anlegen (einmalig) und aktivieren:

```
python -m venv .venv
.venv\Scripts\activate
```

Vor der Eingabezeile steht danach `(.venv)`. Die Aktivierung gilt nur für dieses Fenster; in jedem neuen Fenster ist `.venv\Scripts\activate` erneut nötig.

3. Pakete installieren (einmalig, dauert ein bis zwei Minuten):

   ```
   pip install -r requirements.txt
   ```

4. Installation prüfen:

   ```
   python -m pytest
   ```

   Erwartet: eine Zeile wie `40 passed`, kein `failed`. Der Test wertet die mitgelieferten Messdaten vollständig aus und vergleicht jede Zahl mit den Referenzergebnissen in `tests/referenz/`. Läuft er durch, rechnet die Installation richtig.

Unter Linux und macOS lautet die Aktivierung `source .venv/bin/activate`.

1.5 Erster Start

Mit aktivierter Umgebung im Projektordner:

```
python auswertung.py
```

Das Auswahlfenster öffnet sich in `data/raw/`. Eine oder mehrere Dateien `Auswertung_Masse_….csv` wählen (mehrere mit gedrückter Strg-Taste) und **Öffnen** klicken. Nach etwa 20 Sekunden je Datei stehen Tabellen und Plots in `ergebnisse/` (Kapitel 3).

1.6 Arbeiten in VS Code oder Spyder

Die Skripte lassen sich auch in einer Entwicklungsumgebung starten. Wichtig ist in beiden Fällen: **Der Projektordner muss das Arbeitsverzeichnis sein**, sonst wird `config.py` nicht gefunden.

VS Code

1. VS Code von https://code.visualstudio.com installieren und die Erweiterung *Python* (von Microsoft) hinzufügen.
2. **Datei → Ordner öffnen** → Projektordner.
3. Unten rechts bzw. mit Strg+Umschalt+P → *Python: Select Interpreter* die Umgebung `.venv` wählen.
4. `auswertung.py` öffnen und oben rechts auf ▷ **Run Python File** klicken. Die Ausgabe erscheint im Terminal unten.

Spyder

1. Unter **Extras → Einstellungen → Python-Interpreter** den Interpreter `…\TaylorFlow_Auswertung\.venv\Scripts\python.exe` eintragen. In der Umgebung zusätzlich `pip install spyder-kernels` ausführen.
2. **Ausführen → Konfiguration pro Datei** → *Arbeitsverzeichnis: Verzeichnis der Datei*.

1.7 Typische Fehler

Meldung	Ursache	Abhilfe
`'python' wird nicht erkannt …` oder der Microsoft Store öffnet sich	Python fehlt im PATH	Installer → Modify → *Add Python to environment variables* (1.1)
`ModuleNotFoundError: No module named 'thermo'` (oder `numpy`, `yaml`)	Umgebung nicht aktiviert oder Pakete nicht installiert	`.venv\Scripts\activate`, dann `pip install -r requirements.txt`
`ModuleNotFoundError: No module named 'config'`	Skript nicht aus dem Projektordner gestartet	in den Projektordner wechseln (1.4) bzw. Arbeitsverzeichnis einstellen (1.6)
`ModuleNotFoundError: No module named 'tkinter'`	Python ohne *tcl/tk* installiert	Installer → Modify → *tcl/tk and IDLE* anhaken, oder feste Dateiliste statt Auswahlfenster (Kapitel 3.5)
`Keine Messdatei ausgewählt, Auswertung beendet.`	Auswahlfenster abgebrochen	erneut starten und Datei wählen
`Dateiname '…' passt nicht zum Schema`	Messdatei falsch benannt	Kapitel 2.2
`In '…' fehlen die Spalten […]`	Datei stammt nicht aus der Videoauswertung oder ist unvollständig	Kapitel 2.3
Tests schlagen mit sehr kleinen Abweichungen fehl	andere Paketversionen liefern Stoffdaten, die in der letzten Stelle abweichen	Versionen aus `requirements.txt` verwenden; getestet mit numpy 2.5, pandas 3.0, scipy 1.18, thermo 0.6

2 Messdaten

2.1 Herkunft

Eingangsdaten sind die Ergebnisdateien der Videoauswertung (siehe deren Handbuch, Kapitel 4). Je Kanal und Messart gibt es eine Datei, in der jede Zeile ein Positions-Bin einer Kameraposition bei einem Betriebspunkt ist:

```
Videoauswertung\output\
  Output_MassTransfer\
    Auswertung_Masse_PVC_R_1000um_72cm_20molCO2_600um.csv     →  auswertung.py
  Output_PressureCalibration\
    Auswertung_Druck_PVC_R_1000um_72cm_0molCO2_600um.csv      →  kalibrierung.py
```

Die Videoauswertung erzeugt Dateinamen und Spalten bereits im passenden Format. Die Dateien werden unverändert verwendet und beim Start im Auswahlfenster gewählt (Kapitel 3).

2.2 Dateinamen

Alle Angaben zum Versuchsaufbau werden **ausschließlich aus dem Dateinamen** gelesen; gleichnamige Spalten in der Datei (Material, Kanal_um, …) werden nicht verwendet.

`Auswertung_<Messart>_<Material>_<Form>_<D>um_<L>cm_<X>mol<Gas>_<Nadel>um.csv`

Feld	Bedeutung	Beispiel
Messart	`Masse` = Stoffübergangsmessung, `Druck` = Messung ohne CO2 für die Druckkalibrierung	`Masse`
Material	Kanalmaterial, frei wählbar (Buchstaben und Ziffern)	`PVC`, `Glas`, `PTFE`
Form	`R` oder `Rund` = runder Kanal; `Q` oder `Eckig` = quadratischer Kanal	`R`
D	Kanaldurchmesser bzw. Seitenlänge [µm]	`1000`
L	Reaktorlänge [cm]; legt die Länge des Rechengitters fest	`72`
X	CO2-Anteil im Eingangsgas [mol-%]	`20`
Gas	Gasname; Henry-Konstanten sind für CO2, O2, N2 und Luft hinterlegt	`CO2`
Nadel	Innendurchmesser der Gaseinlassnadel [µm]	`600`

Aus dem Dateinamen entsteht der **Kanalname**. Unter ihm stehen Ergebnisse, Plots und Ausschlussregeln:

```
Datei:      Auswertung_Masse_PVC_R_1000um_72cm_20molCO2_600um.csv
Kanalname:  PVC_Rund_1000um_72cm_20molCO2_600um
```

Zwei Messreihen mit demselben Kanalnamen überschreiben sich gegenseitig. Eine Wiederholungsmessung am selben Aufbau bekommt deshalb ein eigenes Material-Kürzel, z. B. `PVC2`.

Für die Druckkalibrierung gilt ein kürzerer **Kalibrierschlüssel** ohne Länge und CO2-Anteil, z. B. `PVC_circular_1000um_600um`. Eine Druckmessung gilt damit für alle Stoffübergangsmessungen am selben Kanal mit derselben Nadel.

2.3 Dateiinhalt

Format: Semikolon als Trennzeichen, Dezimalpunkt oder Dezimalkomma, Kodierung UTF-8.

Pflichtspalten

Spalte	Einheit	Bedeutung
`GasVelocity`	m/s	Leerrohrgeschwindigkeit Gas u_G (Sollwert)
`LiquidVelocity`	m/s	Leerrohrgeschwindigkeit Flüssigkeit u_L
`CameraPosition`	–	Nummer der Kameraposition (1 = am Einlass)
`BinCenter_mm`	mm	Position des Bins im Kanal, gemessen ab Einlass
`mean_BubbleLength_mm`	mm	mittlere Blasenlänge im Bin
`mean_UnitCellLength_mm`	mm	mittlere Länge der Einheitszelle (Blase + Slug)
`mean_Speed_m_s`	m/s	mittlere Blasengeschwindigkeit
`Temperatur_C`	°C	Temperatur (je Betriebspunkt gemittelt)
`Druck_Pa_Auslass`	Pa	Absolutdruck am Auslass

Zusätzlich verwendet, falls vorhanden

Spalte	Verwendung
`std_BubbleLength_mm`	Messunsicherheit im Blasenlängen-Fit und im Kamerafilter
`GroupCount` bzw. `numel_BubbleLength_mm`	Anzahl Messpunkte im Bin; für den Standardfehler und den Kamerafilter

Fehlt die Anzahl je Bin, entfällt der Kamerafilter (2.5), und der Fit gewichtet mit der Standardabweichung statt mit dem Standardfehler.

Mehrere Aufnahmen oder Sensitivitäten. Enthält eine Datei für dieselbe Kamera und dasselbe Bin mehrere Zeilen, z. B. aus mehreren Aufnahmen oder einem Sensitivitäts-Sweep der Videoauswertung, wird je Kamera und Bin die Zeile mit der **kleinsten Blasenlänge** verwendet. Für eine nachvollziehbare Auswertung sollte die Datei je Betriebspunkt nur eine Sensitivität enthalten.

2.4 Ausschlussregeln

Messpunkte oder ganze Betriebspunkte, die nicht ausgewertet werden sollen, stehen in `config/ausschluesse.yaml`, gegliedert nach Kanalname. Jede Regel nennt ihren Grund. Es gibt drei Arten:

```yaml
PVC_Rund_1000um_72cm_20molCO2_800um:
  messpunkte_entfernen:          # Zeilen der Messdatei vor der Auswertung entfernen
    - GasVelocity: 0.134
      LiquidVelocity: 0.25
      grund: Absorption vor dem ersten Messpunkt abgeschlossen
    - GasVelocity: 0.07
      LiquidVelocity: 0.111
      CameraPosition: 3          # nur Kamera 3 dieses Betriebspunkts
      grund: Kamera 3 unscharf
  betriebspunkte_ueberspringen:  # Betriebspunkt erscheint in keiner Ergebnisdatei
    - LiquidVelocity: 0.056      # alle Betriebspunkte mit u_L = 0,056 m/s
      grund: Pumpe instabil
  kla_ungueltig:                 # wird ausgewertet, aber kLa_log_mean bleibt leer
    Gas_0_07_Liq_0_111: "Absorption kaum auswertbar"
```

Art	Felder	Wirkung
`messpunkte_entfernen`	`GasVelocity`, `LiquidVelocity`, `CameraPosition`, `BinCenter_mm` (Toleranz 0,1 mm); alle angegebenen müssen zutreffen	Zeilen fallen vor der Auswertung weg
`betriebspunkte_ueberspringen`	`GasVelocity` und/oder `LiquidVelocity`; ein fehlendes Feld bedeutet „alle Werte“	Betriebspunkt wird nicht ausgewertet
`kla_ungueltig`	Name des Betriebspunkts wie in Spalte `Config`	kLa_log_mean leer, `manual_exclude = True` mit Grund; Profile und Blasenlängen-Fit bleiben verfügbar, 3D-Flächen und Modellvergleiche lassen den Punkt weg

Die mitgelieferte Datei enthält die Ausschlüsse der mitgelieferten Messreihen mit Begründung. Nach einer Änderung `auswertung.py` erneut ausführen.

2.5 Automatischer Kamerafilter

Vor der Auswertung werden unzuverlässige Kamerapositionen entfernt (nur Messart `Masse`). Ein Bin ist gültig, wenn alle drei Bedingungen erfüllt sind:

- relativer Standardfehler der Blasenlänge ≤ 5 %
- mindestens 50 Messpunkte im Bin
- Variationskoeffizient ≤ 50 %

Eine Kamera eines Betriebspunkts bleibt, wenn mindestens 75 % ihrer Bins und mindestens 3 Bins gültig sind. Betriebspunkte mit nur noch einer Kamera entfallen. Die Konsole meldet, wie viele Bins und Kameras verworfen wurden. Die Grenzwerte stehen in `config.PARAMETER` (`kamerafilter_*`, Kapitel 5).

2.6 Neue Messreihe auswerten

1. In der Videoauswertung den Batch für den Messordner laufen lassen. Die Datei `Auswertung_Masse_….csv` entsteht in `output/Output_MassTransfer/`.
2. `python auswertung.py` starten und die Datei im Auswahlfenster wählen.

3. Ergebnisse prüfen (Kapitel 3.4): Warnungsprotokoll, Blasenlängen-Fits, Kennzahlen-Übersicht.
4. Unbrauchbare Messpunkte oder Betriebspunkte in `config/ausschluesse.yaml` eintragen (2.4) und erneut auswerten.
5. **Runder Kanal:** Ohne Eintrag in der Kalibrier-Datenbank rechnet das Druckmodell mit dem Theoriewert c = 0,465; die Konsole meldet dann „nicht kalibriert“. Mit einer Druckmessung kann der Kanal kalibriert werden (Kapitel 10). **Quadratischer Kanal:** keine Kalibrierung nötig.

Andere Abmessungen (Durchmesser, Länge, CO2-Anteil): Es ist nichts weiter anzupassen; alles kommt aus dem Dateinamen.

Andere Flüssigkeit als Wasser oder **rechteckige, nicht quadratische Kanäle** werden nicht unterstützt (Kapitel 8 und 12).

3 Erste Auswertung

Dieses Kapitel führt einmal durch eine komplette Auswertung, vom Start bis zur Prüfung der Ergebnisse.

3.1 Programmteile im Überblick

Alle Einstellungen stehen in **einer** Datei, `config.py`. Gestartet wird über eines von fünf Skripten im Projektordner:

Skript	Zweck	Kapitel
`auswertung.py`	Messdateien auswerten, Tabellen und Plots erzeugen	3, 4
`abbildung.py`	eine einzelne Abbildung erzeugen und anpassen, z. B. für eine Arbeit	4.11
`literaturvergleich.py`	Messwerte mit Literaturmodellen vergleichen, eigenes Modell anpassen	9
`kalibrierung.py`	Bretherton-Faktor c aus Druckmessungen bestimmen	10
`monolith.py`	Monolith auswerten und mit einem Kanal vergleichen	11

3.2 Starten

In der Eingabeaufforderung (Umgebung aktiviert, Kapitel 1.4):

```
cd C:\Messungen\TaylorFlow_Auswertung
.venv\Scripts\activate
python auswertung.py
```

oder in VS Code `auswertung.py` öffnen und ▷ **Run Python File** klicken.

3.3 Messdateien auswählen

Es öffnet sich das Fenster **„Messdateien auswählen“**. Es startet in dem Ordner, der in `config.py` als `DIALOG_ORDNER` eingetragen ist, und zeigt nur Dateien `Auswertung_Masse_*.csv`. Über die Auswahlliste unten rechts lassen sich auch alle CSV-Dateien anzeigen.

- Eine Datei: anklicken und **Öffnen**.
- Mehrere Dateien: mit gedrückter **Strg**-Taste anklicken, oder mit **Strg+A** alle markieren.
- **Abbrechen** beendet das Programm ohne Auswertung.

Die Konsole zeigt danach die gewählten Dateien und für jeden Betriebspunkt die wichtigsten Zwischenergebnisse:

```
Messdateien:
  C:\…\data\raw\Auswertung_Masse_Glas_Q_1000um_72cm_20molCO2_600um.csv

=== Glas_Eckig_1000um_72cm_20molCO2_600um ===
  200 Messpunkte laut ausschluesse.yaml entfernt.
  Kamerafilter: 0 von 2892 Bins entfernt (0.0 %), 0 von 54 Kamerapositionen.

  Betriebspunkt Gas_0_07_Liq_0_02 (u_G = 0.07 m/s, u_L = 0.02 m/s)
    übersprungen (ausschluesse.yaml): Starke Abweichung des gemessenen Drucks.

  Betriebspunkt Gas_0_07_Liq_0_111 (u_G = 0.07 m/s, u_L = 0.111 m/s)
    Blasenfit: Exp+Expansion v2+Offsets (…), R² = 0.8338, chi²_red = 5.06
    Offset Kamera 2: -0.024 mm
    Offset Kamera 3: -0.019 mm
    Druck: Auslass 100373.3 Pa, Einlass 108958.7 Pa
    kLa_log_mean = 0.285 1/s (721 Punkte in der aktiven Zone)
  …
Plots für Glas_Eckig_1000um_72cm_20molCO2_600um
  gespeichert in C:\…\ergebnisse\Glas_Eckig_1000um_72cm_20molCO2_600um\plots

Fertig nach 18 s.
```

3.4 Ergebnisse finden und prüfen

```
ergebnisse\
  Modell_Gueltigkeitswarnungen.txt          Warnungen dieses Laufs (Kapitel 6.4)
  Glas_Eckig_1000um_72cm_20molCO2_600um\
    …_betriebspunkte.csv                    eine Zeile je Betriebspunkt, u. a.
     ↪  kLa_log_mean
    …_profile.csv                           Verläufe entlang des Kanals
    …_meta.json                             Quelldatei und Zeitpunkt
    plots\                                  alle Plots (Kapitel 4)
```

Die CSV-Dateien lassen sich direkt in Excel öffnen (Semikolon, Dezimalkomma). Alle Spalten erklärt Kapitel 6.

Bei einer neuen Messreihe in dieser Reihenfolge prüfen:

1. **Warnungsprotokoll** `Modell_Gueltigkeitswarnungen.txt` lesen (Bedeutung der Meldungen: Kapitel 6.4).
2. **Blasenlängen-Fit** jedes Betriebspunkts ansehen (Dateien `blasenlaenge_*.png` in `plots`, Kapitel 4.2): Folgt die dunkelblaue Kurve den Messpunkten aller Kameras? Fällt die grüne Kurve monoton ab?
3. **Kennzahlen-Übersicht** `plots\heatmaps.png` auf Ausreißer prüfen.
4. Unbrauchbare Punkte in `config/ausschluesse.yaml` eintragen (Kapitel 2.4) und erneut auswerten.

3.5 Häufige Varianten

Nur Plots neu zeichnen, z. B. nach einer Änderung in `PLOTS` oder `STIL`: in `config.py`

```
NEU_RECHNEN = False
```

setzen und `auswertung.py` erneut starten. Im Auswahlfenster dieselben Dateien wählen; es wird nichts neu gerechnet, das Zeichnen dauert nur Sekunden.

Ohne Auswahlfenster, z. B. für wiederholte Läufe mit denselben Dateien:

```
MESSDATEIEN = [
    Path(r"C:\Messungen\Videoauswertung\output\Output_MassTransfer\Auswertung_
↪   Masse_PVC_R_1000um_72cm_20molCO2_600um.csv"),
    Path(r"C:\Messungen\Videoauswertung\output\Output_MassTransfer\Auswertung_
↪   Masse_Glas_Q_1000um_72cm_20molCO2_600um.csv"),
]
```

oder `MESSDATEIEN = "alle"` für alle Dateien `Auswertung_Masse_*.csv` in `ROHDATEN_ORDNER` (Standard `data/raw/`).

Plots nur für einzelne Betriebspunkte:

```
BETRIEBSPUNKTE = ["Gas_0_134_Liq_0_056", "Gas_0_198_Liq_0_111"]
```

Die Namen stehen in der Spalte `Config` der Betriebspunkt-Tabelle und in der Konsolenausgabe.

4 Plots auswählen und anpassen

4.1 Grundprinzip

`auswertung.py` erzeugt nach der Rechnung alle eingeschalteten Plots und legt sie in `ergebnisse/<Kanal>/plots/` ab. Was erscheint und wie es aussieht, steht an zwei Stellen in `config.py`:

Abschnitt	Gilt für	Beispiele
`PLOTS`	je Plot ein eigener Block	ein/aus, Achsbereiche, Bildelemente, Farben, Farbskala, Heatmap-Felder
`STIL`	alle Plots gemeinsam	Sprache, Dateiformat, Auflösung, Schriftgröße, Standardgrößen, Titel ein/aus

Jeder Block in `PLOTS` führt **alle** möglichen Einträge mit ihrem Standardwert auf. Zum Anpassen wird nur der Wert geändert:

```
PLOTS = dict(
    blasenlaengen_fit=dict(
        an=True, titel="auto", groesse_cm=None,
        x_bereich_cm=None,              # Reaktorposition z [cm]
        y_bereich_mm=None,              # Blasenlänge [mm]
        ...
```

Nach einer Änderung genügt ein Lauf mit `NEU_RECHNEN = False` (Kapitel 3.5); die Plots sind dann in Sekunden neu gezeichnet.

Für alle Plots gleich:

Eintrag	Bedeutung	Beispiel
`an`	Plot erzeugen	`an=False`
`titel`	`"auto"` = Kanalname (bzw. Kanal und Betriebspunkt), eigener Text oder `""` für ohne Titel	`titel="PVC, 1 mm"`
`groesse_cm`	(Breite, Höhe) in cm; `None` = Standardgröße aus `STIL`	`groesse_cm=(9, 7)`
Achsbereiche	`(min, max)` in der Einheit der Achse; `None` = automatisch	`x_bereich_cm=(0, 40)`
Farben	Name (`"tab:red"`, `"black"`) oder Hex-Code (`"#3B8B3B"`)	`farbe="tab:red"`
Farbskalen	Name einer matplotlib-Farbskala	`farbskala="viridis"`

Gängige Farbskalen: `"plasma"`, `"viridis"`, `"cividis"`, `"magma"`, `"coolwarm"`, `"RdYlGn"`, `"Blues"`. Mit der Endung `_r` wird die Skala umgekehrt (`"viridis_r"`).

Tippfehler werden sofort gemeldet, mit allen möglichen Namen:

```
KeyError: "Unbekannte Option ['farbskale'] für den Plot 'kla_flaeche'.
Möglich: ['achsen', 'an', 'blickwinkel', 'farbskala', 'groesse_cm', ...]"
```

Übersicht der Plots

Block in `PLOTS`	Plot	Datei	Abschnitt
`blasenlaengen_fit`	Blasenlänge entlang des Kanals mit Fit	`blasenlaenge_<Betriebspunkt>.png`	4.2
`stoffmenge`	absorbierte CO2-Menge je Blase	`stoffmenge_<Betriebspunkt>.png`	4.3
`druckprofil`	berechneter Druck entlang des Kanals	`druck_<Betriebspunkt>.png`	4.4
`kla_flaeche`	kLa_log_mean über u_G und u_L (3D)	`kla_flaeche.png`	4.5
`nspez_flaeche`	volumenspezifischer Molenstrom (3D)	`nspez_flaeche.png`	4.6
`modell_flaechen`	Vorhersage von Literatur- oder eigenen Modellen (3D)	`modell_<Modell>.png`	4.7
`spalten_flaechen`	beliebige Spalte der Ergebnistabelle (3D), standardmäßig aus	`flaeche_<Spalte>.png`	4.8
`heatmaps`	Kennzahlen-Übersicht als Farbkarten	`heatmaps.png`	4.9

Die ersten drei Plots entstehen **je Betriebspunkt**; welche Betriebspunkte, legt `BETRIEBS-PUNKTE` fest (Kapitel 3.5). Die übrigen entstehen **je Kanal**.

4.2 Blasenlängen-Fit (`blasenlaengen_fit`)

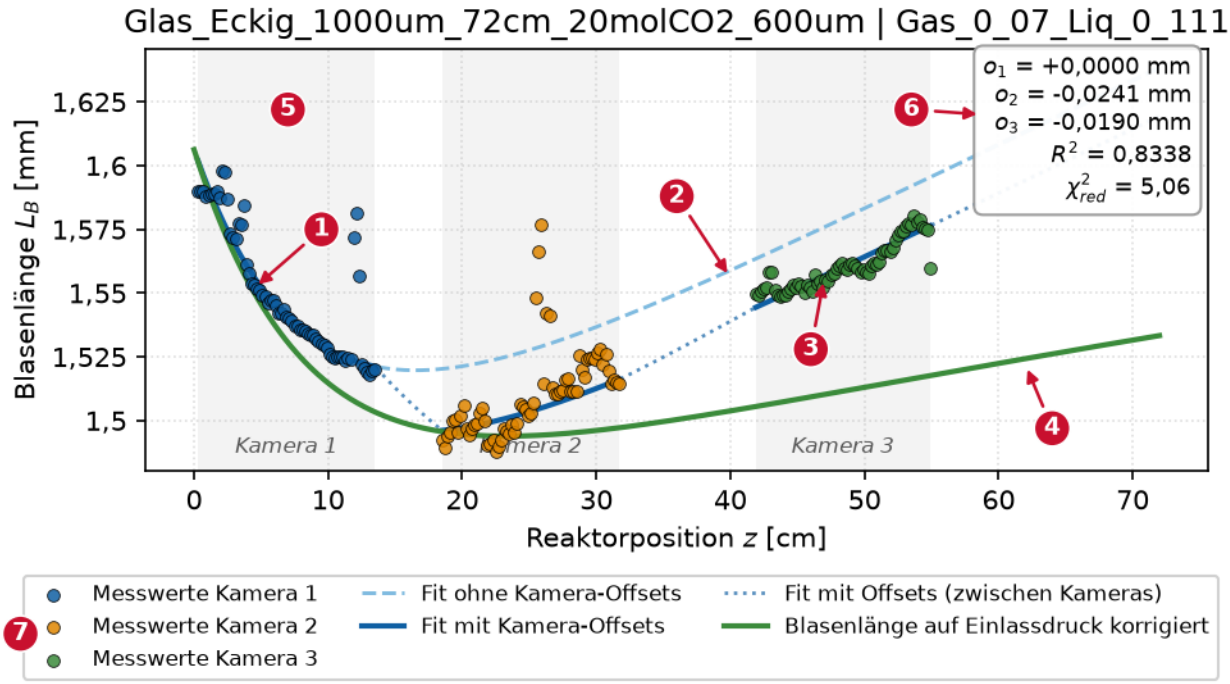


Abbildung 1: Blasenlängen-Fit mit den Bildelementen, Glas eckig 1000 µm, u_G = 0,07 m/s, u_L = 0,111 m/s

Nr.	Bildelement	Bedeutung	Eintrag
1	Punkte, farbig je Kamera	gemessene Blasenlängen je Bin	`messwerte`
2	hellblau gestrichelt	Fit **ohne** Kamera-Offsets; mit dieser Kurve wird weitergerechnet	`fit_ohne_offsets`
3	dunkelblau, zwischen den Kameras gepunktet	derselbe Fit plus Offset der jeweiligen Kamera; muss den Punkten folgen	`fit_mit_offsets`
4	grün	Blasenlänge auf den Einlassdruck umgerechnet; enthält nur noch die Absorption	`druckkorrigiert`
5	graue Bänder	Bereich der Messpunkte je Kamera	`kamerafenster`
6	Kasten	Kamera-Offsets o_j, R^2 und χ^2_red des Fits	`kennwertkasten`
7	Legende	Erklärung der Linien	`legende`

Worauf achten:

- Die dunkelblaue Kurve folgt den Punkten aller Kameras.
- Die grüne Kurve fällt und flacht ab. Ein Wiederanstieg weist auf ein zu niedrig berechnetes Druckprofil hin.
- Die Offsets betragen wenige hundertstel Millimeter. Bei Offsets in der Größe der Absorption warnt das Protokoll.

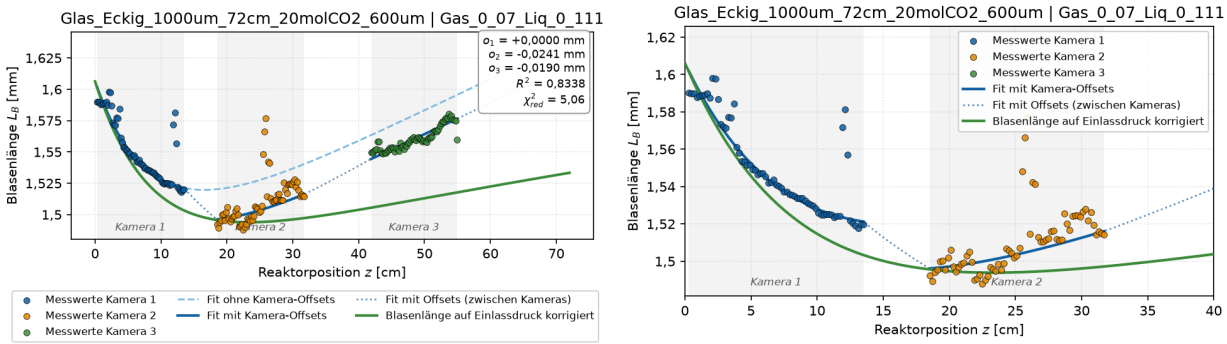
- χ^2_red über etwa 3 heißt, dass das Modell die Streuung nicht vollständig erklärt. Einzelne Ausreißer (im Bild bei z ≈ 12 cm und 26 cm) dämpft der robuste Fit.

Anpassen

Wunsch	Eintrag im Block `blasenlaengen_fit`
nur den vorderen Kanalabschnitt zeigen	`x_bereich_cm=(0, 40)`
feste y-Achse für mehrere Betriebspunkte	`y_bereich_mm=(1.4, 1.7)`
ohne den gestrichelten Fit	`fit_ohne_offsets=False`
nur Messwerte und Fit, ohne Druckkorrektur	`druckkorrigiert=False`
ohne graue Kamerabänder	`kamerafenster=False`
ohne Kasten mit Kennwerten	`kennwertkasten=False`
Legende im Plot statt darunter	`legende="innen"` (freie Stelle) oder z. B. `legende="upper right"`
ohne Legende	`legende="aus"`

Mögliche Legendenpositionen: `"upper left"`, `"upper center"`, `"upper right"`, `"center left"`, `"center"`, `"center right"`, `"lower left"`, `"lower center"`, `"lower right"`.

Beispiel: links der Standard, rechts mit `x_bereich_cm=(0, 40), fit_ohne_offsets=False, kennwertkasten=False, legende="upper right"`.

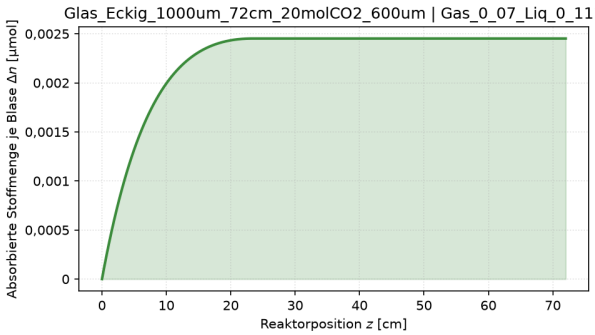


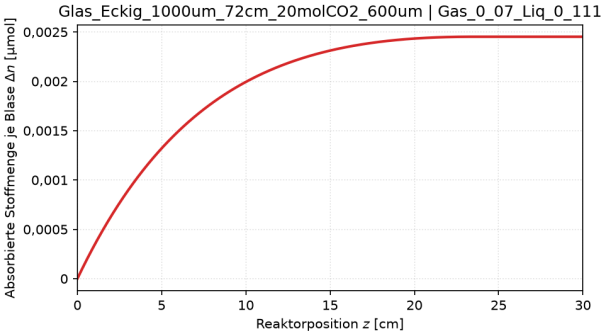
4.3 Absorbierte Stoffmenge (`stoffmenge`)

Kumuliert absorbierte CO2-Menge je Blase in µmol über der Kanalposition. Der Verlauf flacht ab, sobald die Flüssigkeit gesättigt ist oder das CO2 verbraucht ist.

Wunsch	Eintrag im Block `stoffmenge`
Achsbereiche	`x_bereich_cm=(0, 30)`, `y_bereich_umol=(0, 0.5)`
ohne eingefärbte Fläche unter der Kurve	`flaeche=False`
andere Linienfarbe	`farbe="tab:red"`

Links der Standard, rechts mit `flaeche=False, farbe="tab:red", x_bereich_cm=(0, 30)`:



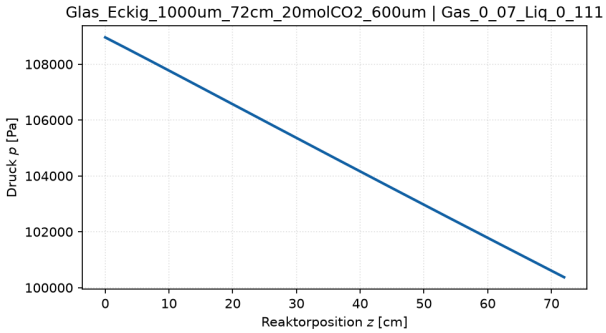


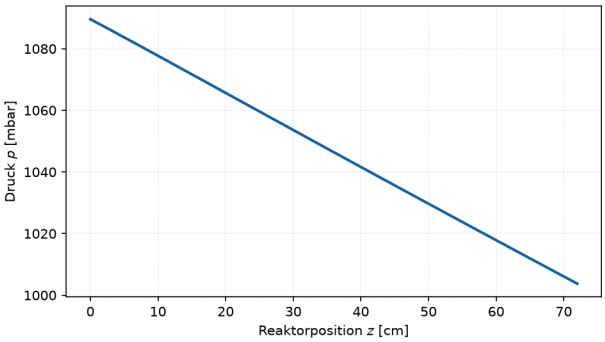
4.4 Druckprofil (`druckprofil`)

Berechneter Absolutdruck, vom gemessenen Auslassdruck rückwärts integriert. Die Steigung ist der Druckgradient des Bretherton- bzw. Kurimoto-Modells (Kapitel 7.4).

Wunsch	Eintrag im Block `druckprofil`
Druck in mbar, hPa, kPa oder bar	`einheit="mbar"`
Achsbereiche (y in der gewählten Einheit)	`x_bereich_cm=(0, 72)`, `y_bereich=(1000, 1100)`
andere Linienfarbe	`farbe="black"`

Links der Standard, rechts mit `einheit="mbar", titel=""`:





4.5 kLa-Fläche (`kla_flaeche`)

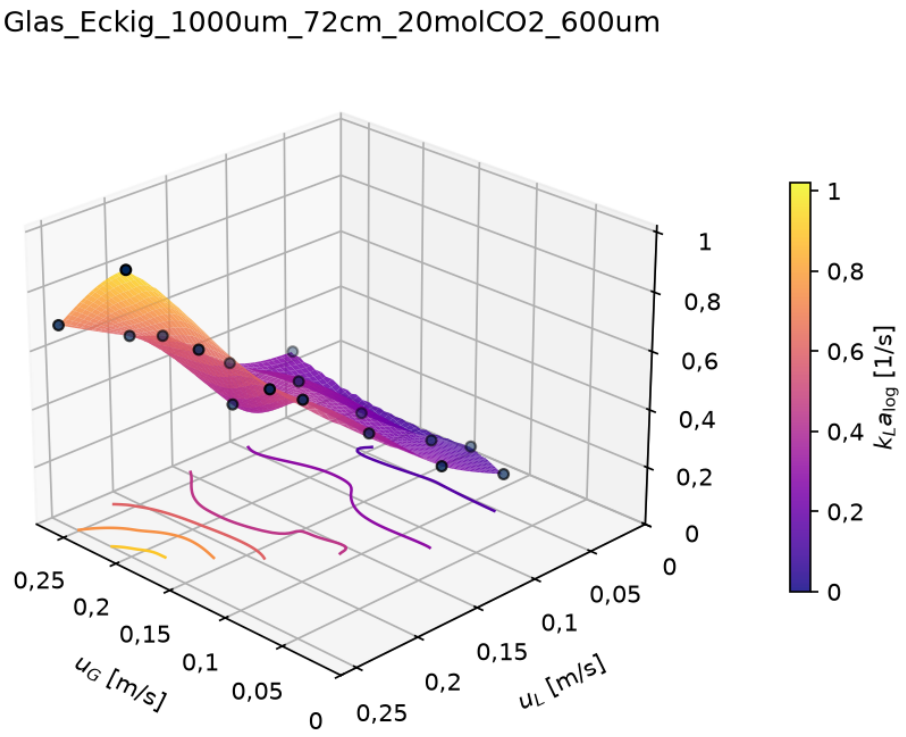


Abbildung 2: kLa_log_mean über u_G und u_L, Glas eckig 1000 µm

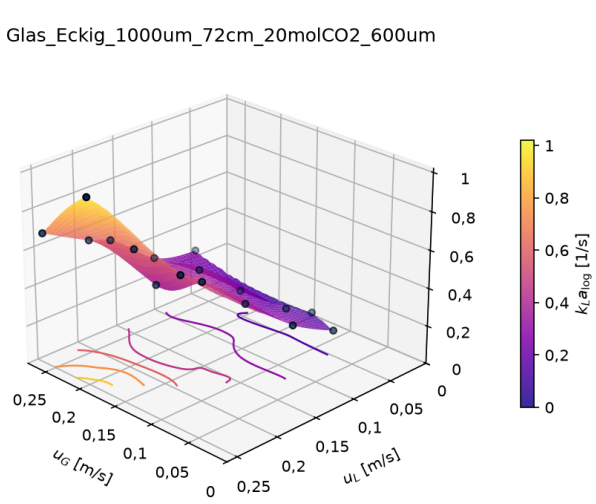
- **Fläche:** kLa_log_mean über u_G und u_L, kubisch zwischen den Messpunkten interpoliert; außerhalb der Messpunkte bleibt sie leer.
- **Punkte:** die Messwerte. Bei wenigen, unregelmäßig verteilten Punkten kann die Interpolation über- oder unterschwingen; maßgeblich sind die Punkte.
- **Linien am Boden:** Höhenlinien.
- Betriebspunkte ohne gültigen kLa-Wert (`kla_ungueltig`) fehlen.

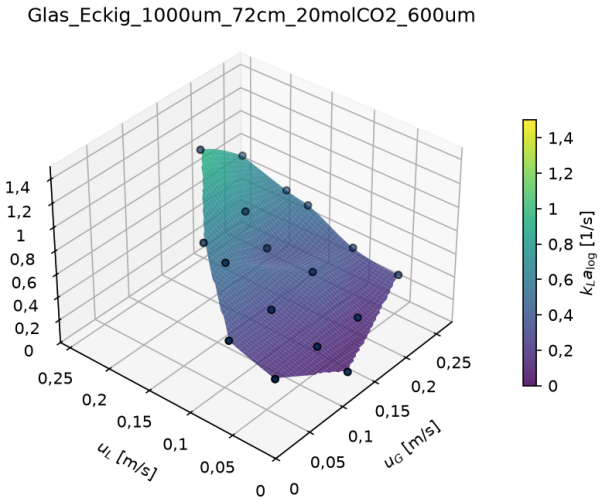
Anpassen (gilt genauso für alle 3D-Flächen, Abschnitte 4.6 bis 4.8)

Wunsch	Eintrag
andere Farbskala	`farbskala="viridis"`
feste Achsbereiche, z. B. für den Vergleich mehrerer Kanäle	`achsen=dict(x=(0, 0.3), y=(0, 0.3), z=(0, 1.5))`; nicht genannte Achsen bleiben automatisch
andere Blickrichtung	`blickwinkel=(35, 220)`: Höhe und Drehung in Grad
ohne Messpunkte	`messpunkte=False`
ohne Höhenlinien	`hoehenlinien=False`
Fläche deckend bzw. durchsichtiger	`transparenz=1.0` bzw. `0.5`

Für die Blickrichtung: Höhe 0° ist die Seitenansicht, 90° die Draufsicht; die Drehung dreht den Plot um die senkrechte Achse (Standard `(25, 135)`).

Links der Standard, rechts mit `farbskala="viridis", blickwinkel=(35, 220), achsen=dict(z=(0, 1.5)), hoehenlinien=False`:

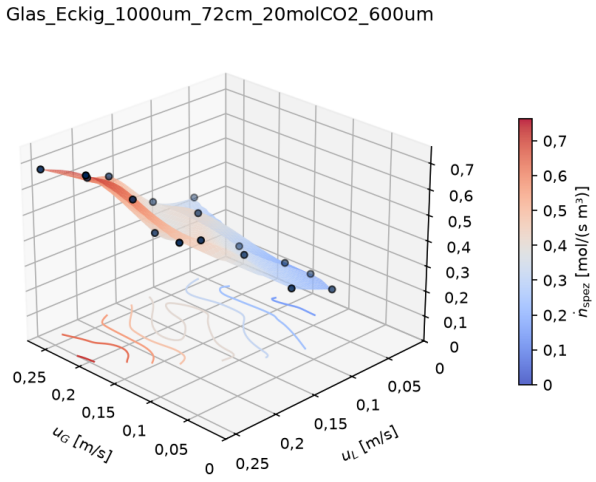


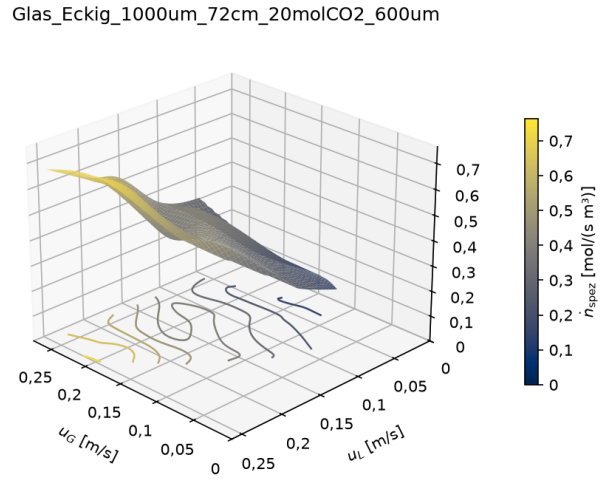


4.6 Volumenspezifischer Molenstrom (`nspez_flaeche`)

ṅ_spez [mol/(s·m³)]: absorbierter CO2-Molenstrom, bezogen auf das gesamte Kanalvolumen (Definition in Kapitel 7.7). Der Bezug auf den ganzen Kanal macht den Wert mit dem Monolithen vergleichbar; gesättigte Abschnitte zählen deshalb mit. Optionen wie in 4.5.

Links der Standard, rechts mit `farbskala="cividis", messpunkte=False, transparenz=1.0`:





4.7 Modellflächen (`modell_flaechen`)

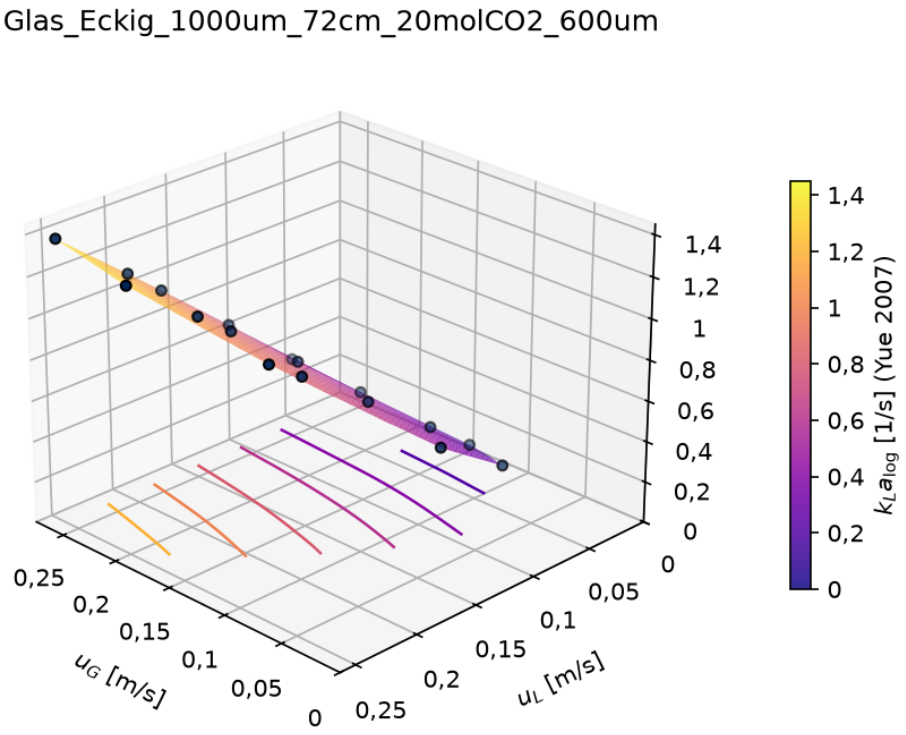


Abbildung 3: Vorhersage von Yue 2007 für den Glaskanal

Vorhersage eines Literatur- oder eigenen Modells für dieselben Betriebspunkte, je Modell ein Bild. Die Modelle stehen in der Liste `modelle`:

```
modell_flaechen=dict(an=True, modelle=["yue_2007", "vandu"], ...)
```

Mögliche Namen: `van_baten_voll`, `van_baten_kappe`, `van_baten_film`, `vandu`, `shao`, `liu_wang`, `heiszwolf`, `yao`, `yue_2007`, `yue_2009` und alle eigenen Modelle aus `config/eigene_modelle.yaml` (Kapitel 9). Modelle, die für die Kanalform nicht gelten (Yue nur eckig, Liu-Wang nur rund), werden mit einem Hinweis übersprungen. Übrige Optionen wie in 4.5.

4.8 Beliebige Spalten als Fläche (`spalten_flaechen`)

Jede Zahlenspalte der Betriebspunkt-Tabelle (Kapitel 6.2) lässt sich als 3D-Fläche über u_G und u_L zeichnen. Der Block ist standardmäßig ausgeschaltet. Je Eintrag in `spalten` entsteht ein Bild `flaeche_<Spalte>.png`:

```
spalten_flaechen=dict(
    an=True, ...,
    spalten=[
        dict(spalte="SlugLength_mean_m", beschriftung="Sluglänge $L_S$ [mm]",
         ↪  faktor=1000),
        dict(spalte="slip_meas_over_theo", beschriftung="Schlupf [-]"),
    ],
)
```

Eintrag je Spalte	Bedeutung
`spalte`	Spaltenname wie in der Tabelle
`beschriftung`	Text an der Farbskala; ohne Angabe der Spaltenname. Formeln in `$…$`
`faktor`	Umrechnung vor dem Zeichnen, z. B. `1000` für m → mm

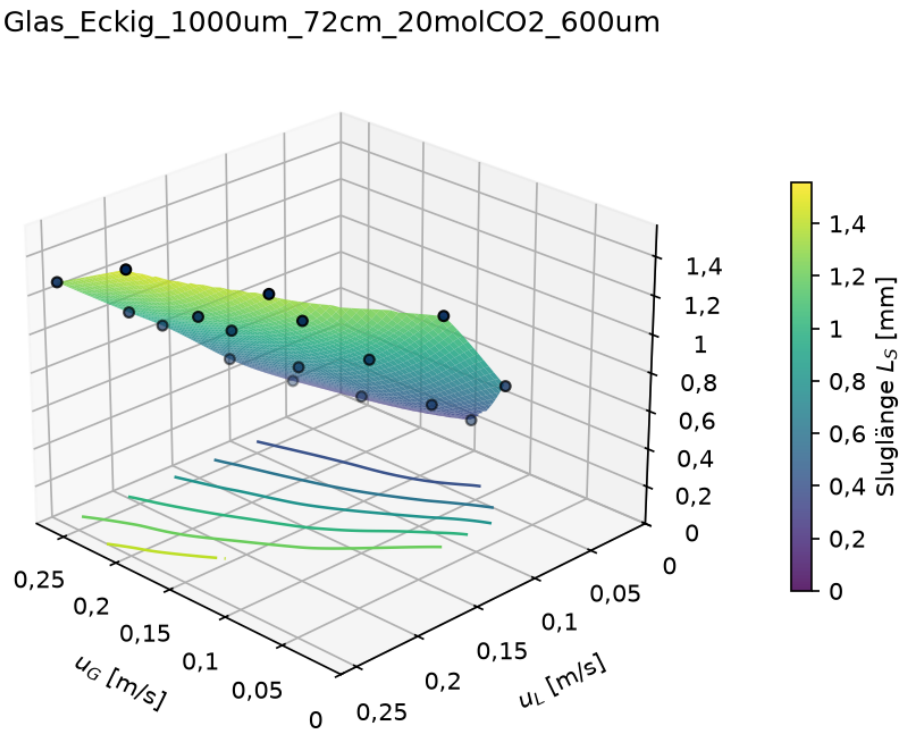


Abbildung 4: Sluglänge als Fläche

4.9 Kennzahlen-Übersicht (heatmaps)

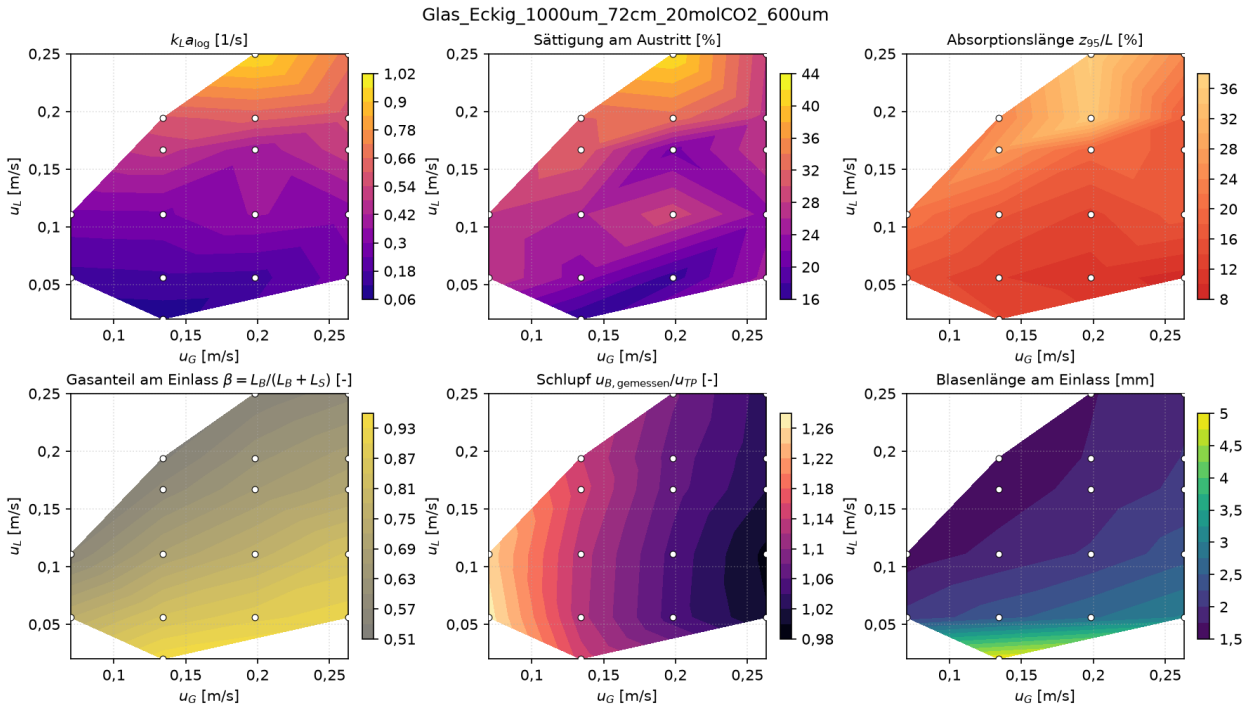


Abbildung 5: Kennzahlen-Übersicht, Glas eckig 1000 µm

Mehrere Größen als Farbkarten über u_G × u_L; die weißen Punkte sind die Betriebspunkte. Die Übersicht eignet sich, um eine neue Messreihe schnell auf Ausreißer zu prüfen. Standardfelder in Leserichtung:

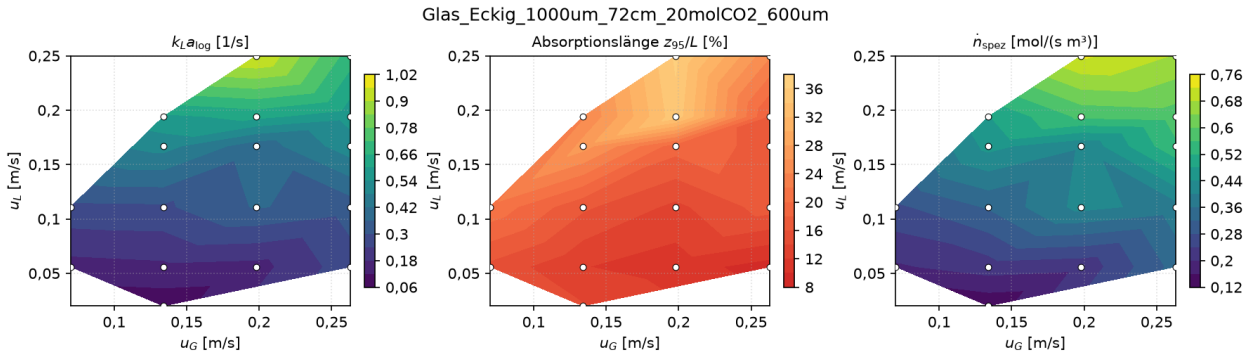
Name in `felder`	Größe
`"kla"`	kLa_log_mean
`"saettigung_aus"`	Sättigung C_L/C* am Kanalende
`"z95"`	Absorptionslänge: Position, an der 95 % der insgesamt absorbierten CO2-Menge aufgenommen sind, bezogen auf die Kanallänge
`"beta_ein"`	Gasanteil der Einheitszelle am Einlass, β = L_B/(L_B + L_S)
`"schlupf"`	gemessene Blasengeschwindigkeit / (u_G + u_L)
`"L_b_ein"`	gefittete Blasenlänge am Einlass

Anpassen

Wunsch	Eintrag im Block `heatmaps`
andere Auswahl oder Reihenfolge	`felder=["kla", "z95", "schlupf"]`; bis drei Felder je Zeile
eine Spalte der Betriebspunkt-Tabelle als Feld	`felder=["kla", "n_spez_mol_s_m3"]`
Überschrift eines Felds	`beschriftungen={"n_spez_mol_s_m3": "ṅ_spez [mol/(s m³)]"}`

Wunsch	Eintrag im Block `heatmaps`
Farbskala eines Felds	`farbskalen=dict(kla="viridis")`

Beispiel mit drei Feldern: `felder=["kla", "z95", "n_spez_mol_s_m3"]`, eigene Überschrift für `n_spez_mol_s_m3` und `farbskalen=dict(kla="viridis")`:

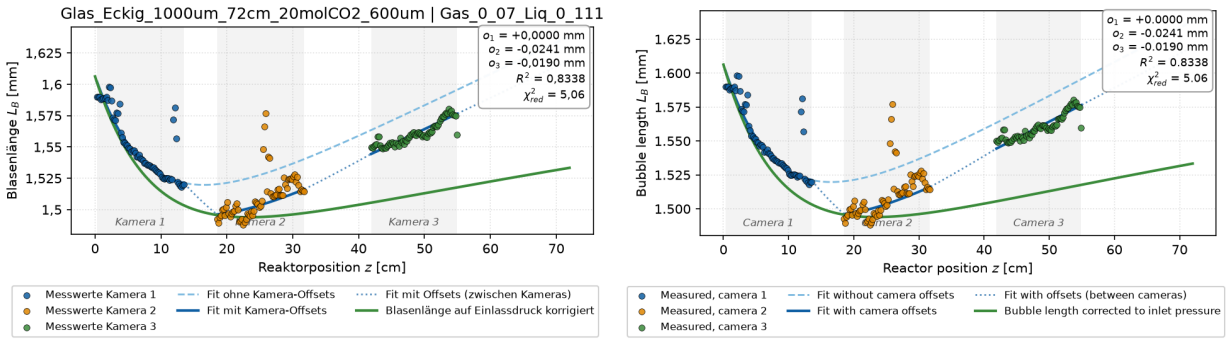


4.10 Darstellung aller Plots (`STIL`)

Eintrag	Standard	Bedeutung
`sprache`	`"de"`	`"de"` (Dezimalkomma) oder `"en"` (Dezimalpunkt, englische Beschriftung)
`formate`	`("png",)`	ein oder mehrere Dateiformate, z. B. `("png", "pdf")`; PDF und SVG sind verlustfrei skalierbar
`dpi`	300	Auflösung der PNG-Dateien
`schriftgroesse`	10	Grundschriftgröße in pt
`groesse_profil_cm`	(16, 9)	Standardgröße der Plots je Betriebspunkt
`groesse_3d_cm`	(14, 11)	Standardgröße der 3D-Flächen
`groesse_heatmaps_cm`	(30, 17)	Standardgröße der Kennzahlen-Übersicht (für 2 × 3 Felder)
`groesse_paritaet_cm`	(12, 11)	Paritätsdiagramme (Kapitel 9 und 11)
`titel_anzeigen`	`True`	`False` schaltet alle Titel ab, z. B. für Abbildungen in Arbeiten
`blickwinkel_3d`	(25, 135)	Standard-Blickrichtung der 3D-Flächen

Der `STIL` gilt auch für `literaturvergleich.py`, `monolith.py` und `kalibrierung.py`. Eine Größe im Block eines Plots (`groesse_cm`) hat Vorrang vor der Standardgröße.

Links deutsch mit Titel (Standard), rechts mit `sprache="en", titel_anzeigen=False`:



4.11 Eine einzelne Abbildung erzeugen (`abbildung.py`)

Für eine Abbildung in einer Arbeit oder einem Vortrag braucht man oft genau einen Plot in einer bestimmten Form, ohne `config.py` dauerhaft zu ändern. Dafür gibt es `abbildung.py`. Oben in der Datei stehen nur wenige Einstellungen:

```
KANAL = "Glas_Eckig_1000um_72cm_20molCO2_600um"   # Ordnername in ergebnisse/
PLOT = "blasenlaengen_fit"                        # Name eines Blocks in config.PLOTS
BETRIEBSPUNKT = "Gas_0_134_Liq_0_056"             # nur für Plots je Betriebspunkt

ANPASSUNGEN = dict(                               # Änderungen gegenüber
 ↪  config.PLOTS[PLOT]
    x_bereich_cm=(0, 40),
    kennwertkasten=False,
    legende="upper right",
)
STIL_ANPASSUNGEN = dict(                          # Änderungen gegenüber config.STIL
    sprache="en",
    formate=("png", "pdf"),
    titel_anzeigen=False,
)
ZIEL_ORDNER = config.ERGEBNIS_ORDNER / "abbildungen"
DATEINAME = None        # None = Name wie in auswertung.py
VORSCHAU = True         # Abbildung zusätzlich in einem Fenster anzeigen
```

Dann `python abbildung.py` ausführen. Die Abbildung wird gespeichert und mit `VORSCHAU = True` in einem Fenster gezeigt. In diesem Fenster lässt sie sich vergrößern und verschieben; gespeichert ist die Fassung mit den Einstellungen aus der Datei.

- Es wird nichts neu gerechnet. `auswertung.py` muss für den Kanal schon gelaufen sein.
- In `ANPASSUNGEN` sind alle Einträge des jeweiligen Blocks aus `config.PLOTS` erlaubt (Abschnitte 4.2 bis 4.9).
- Für `modell_flaechen` und `spalten_flaechen` entsteht je Modell bzw. Spalte ein Bild.

4.12 Rezepte

Abbildung für eine Arbeit (schmal, englisch, Vektorgrafik). In `abbildung.py`:

```
PLOT = "kla_flaeche"
ANPASSUNGEN = dict(groesse_cm=(8, 7), farbskala="viridis")
STIL_ANPASSUNGEN = dict(sprache="en", formate=("pdf",), titel_anzeigen=False,
 ↪  schriftgroesse=8)
```

Mehrere Kanäle auf gleichen Achsen vergleichen. In `config.py` für die Fläche feste Achsen setzen, dann alle Kanäle auswerten bzw. neu zeichnen:

```
kla_flaeche=dict(..., achsen=dict(x=(0, 0.3), y=(0, 0.3), z=(0, 1.5)), ...)
```

Nur die Übersichtsplots, keine Plots je Betriebspunkt. In `config.py` `an=False` in den Blöcken `blasenlaengen_fit`, `stoffmenge` und `druckprofil`.

Eine weitere Größe als Fläche. In `spalten_flaechen` `an=True` setzen und die Spalte in `spalten` eintragen (4.8). Die verfügbaren Spalten listet Kapitel 6.2.

Kennzahlen-Übersicht mit eigener Auswahl. `felder` im Block `heatmaps` (4.9).

4.13 Plots direkt aus Python

Alle Plotfunktionen lassen sich auch selbst aufrufen. Sie geben eine matplotlib-Figure zurück, die sich vor dem Speichern beliebig weiter bearbeiten lässt:

```python
import config
from taylorflow import ergebnisse, plots

ordner = config.ERGEBNIS_ORDNER
kanal = "Glas_Eckig_1000um_72cm_20molCO2_600um"
bp = ergebnisse.lade_betriebspunkte(ordner, kanal)
profile = ergebnisse.lade_profile(ordner, kanal)
stil = plots.PlotStil(**config.STIL).mit(sprache="en")

fig = plots.kla_flaeche(bp, stil, titel="", farbskala="viridis", achsen=dict(z=(0,
 ↪  1.5)))
fig.axes[0].set_zlabel("kLa [1/s]")              # beliebige matplotlib-Änderung
plots.speichere(fig, "abbildungen", "kla_glas", stil)

profil = ergebnisse.profil_eines_betriebspunkts(profile, "Gas_0_134_Liq_0_056")
zeile = bp[bp["Config"] == "Gas_0_134_Liq_0_056"].iloc[0]
fig = plots.blasenlaengen_fit(profil, zeile, stil, x_bereich_cm=(0, 40),
 ↪  legende="innen")
plots.speichere(fig, "abbildungen", "blasen_glas", stil)
```

Die Parameter jeder Funktion entsprechen den Einträgen der Blöcke in `config.PLOTS`; `help(plots.blasenlaengen_fit)` zeigt sie an.

5 Konfiguration

Alle Einstellungen stehen in `config.py` im Projektordner. Die Datei ist in nummerierte Abschnitte gegliedert. Plots und Darstellung (Abschnitte 4 und 5 der Datei) beschreibt Kapitel 4; dieses Kapitel beschreibt die übrigen Einträge.

Werte, die das Rechenergebnis beeinflussen, stehen in `PARAMETER`. Mit ihren Standardwerten entstehen die Referenzergebnisse in `tests/referenz/`. Die Spalte **Wirkung** gibt an, wie stark sich eine Änderung auf kLa_log_mean auswirkt. Grundlage sind Variationsrechnungen mit den mitgelieferten Messreihen (5 Kanäle, 55 Betriebspunkte); „–“ heißt nicht gesondert untersucht.

5.1 Ordner und Dateien

Eintrag	Standard	Bedeutung
`ROHDATEN_ORDNER`	`data/raw`	mitgelieferte Messdateien; Quelle für `MESSDATEIEN = "alle"`
`DIALOG_ORDNER`	wie `ROHDATEN_ORDNER`	Ordner, in dem das Auswahlfenster startet, z. B. der Ausgabeordner der Videoauswertung
`ERGEBNIS_ORDNER`	`ergebnisse`	Ausgaben; ein Unterordner je Kanal
`AUSSCHLUSS_DATEI`	`config/ausschluesse.yaml`	Ausschlussregeln (Kapitel 2.4)
`KALIBRIER_DATENBANK`	`data/calibrated_c_database.json`	Bretherton-Faktoren der runden Kanäle (Kapitel 10)
`MODELL_DATEI`	`config/eigene_modelle.yaml`	gespeicherte eigene Modelle (Kapitel 9)

Pfade außerhalb des Projektordners werden mit `Path(r"…")` angegeben, z. B. `DIALOG_ORDNER = Path(r"D:\Videoauswertung\output\Output_MassTransfer")`.

5.2 Umfang der Auswertung

Eintrag	Standard	Bedeutung
`MESSDATEIEN`	`"dialog"`	`"dialog"` = Auswahlfenster; `"alle"` = alle `Auswertung_Masse_*.csv` in `ROHDATEN_ORDNER`; oder eine Liste von Dateipfaden
`NEU_RECHNEN`	`True`	`False`: nur die Plots aus vorhandenen Ergebnissen neu zeichnen
`BETRIEBSPUNKTE`	`None`	Betriebspunkte für die Plots je Betriebspunkt; `None` = alle

5.3 Auswerte-Parameter (`PARAMETER`)

5.3.1 Kamerafilter

Eintrag	Standard	Einheit	Bedeutung
`kamerafilter_rel_se_max_prozent`	5,0	%	max. relativer Standardfehler der Blasenlänge je Bin
`kamerafilter_n_min`	50	–	min. Anzahl Messpunkte je Bin
`kamerafilter_cv_max_prozent`	50,0	%	max. Variationskoeffizient je Bin
`kamerafilter_anteil_min`	0,75	–	min. Anteil gültiger Bins, damit eine Kamera bleibt
`kamerafilter_bins_min`	3	–	min. Anzahl gültiger Bins je Kamera

5.3.2 Blasenlängen-Fit

Eintrag	Standard	Einheit	Bedeutung	Wirkung
`fit_serien_offsets`	`True`	–	Kamera-Offsets o_j mitfitten (Kamera 1 ist Bezug)	–
`fit_loss`	`"soft_l1"`	–	robuste Verlustfunktion; `"linear"` = klassische kleinste Quadrate	–
`sigma_sys_m`	2e-6	m	systematischer Messfehler, quadratisch zum Standardfehler addiert	–
`sigma_sem_floor_m`	1e-6	m	Untergrenze des Standardfehlers je Bin	–
`fit_lambda0_m`	0,05	m	Startwert der Absorptionslänge λ	keine (0,01–0,2 m)
`offset_bound_anteil`	0,2	–	Grenze $\|o_j\| \leq$ Anteil · mittlere Blasenlänge	keine (Grenze nie erreicht)

Die Kamera löst etwa 55 µm/px auf. Die beiden Fehlergrenzen von 1 und 2 µm liegen weit darunter; sie verhindern nur, dass Bins mit sehr vielen Blasen den Fit dominieren.

5.3.3 Integration

Eintrag	Standard	Einheit	Bedeutung	Wirkung
`dz_m`	1e-3	m	Ortsschritt für Druckprofil und Stoffbilanz	gering (0,25–2 mm: Prozentbereich)
`u_fit_cap_faktor`	1,5	–	gefittete Geschwindigkeiten über Faktor · U_ref werden verworfen und interpoliert; U_ref = max(Median der Messung, u_G + u_L)	keine (nie aktiv)

Eintrag	Standard	Einheit	Bedeutung	Wirkung
`saettigung_clip`	0,999	–	C_L wird höchstens auf diesen Anteil von C* gesetzt	keine (0,99–0,999)

5.3.4 Messunsicherheiten

Eintrag	Standard	Bedeutung
`u_rel_h_cp`	0,03	relative Unsicherheit der Henry-Konstante
`u_rel_p_co2`	0,02	relative Unsicherheit des CO2-Partialdrucks

Beide Werte legen fest, wo die aktive Zone endet (Kapitel 7.6). Die übrigen drei Einträge dieses Abschnitts (`k_opt_mm_pro_px`, `u_rel_dn_dt`, `pixel_unsicherheit_px`) gehören zur Fehlerfortpflanzung in `taylorflow/unsicherheit.py`. Diese Funktion ist für eigene Unsicherheitsbetrachtungen vorhanden und wird von der Auswertung nicht aufgerufen.

5.3.5 kLa-Auswertung

Eintrag	Standard	Bedeutung	Wirkung
`max_urel_delta_c`	0,5	ein Gitterpunkt gehört zur aktiven Zone, wenn ΔC > 0 und die Unsicherheit von C* höchstens dieser Anteil von ΔC ist	keine (0,3–0,7)
`logmean_min_aktive_punkte`	10	weniger aktive Gitterpunkte: kLa_log_mean = 0	–

5.3.6 Druckverlust und Stoffdaten

Eintrag	Standard	Bedeutung
`c_bretherton_theorie`	0,465	Bretherton-Faktor für runde Kanäle ohne verwendbare Kalibrierung
`fluessigkeit`	`"Water"`	Flüssigkeitsname für die Bibliothek `thermo`

Die Literaturmodelle rechnen mit festen Stoffwerten für Wasser/CO2 bei etwa 25 °C (Kapitel 9.2). Diese stehen in `taylorflow/parameter.py` (`lit_*`) und lassen sich bei Bedarf ebenfalls in `PARAMETER` setzen.

Ein unbekannter Name in `PARAMETER` führt zu einer Meldung mit allen erlaubten Namen.

5.4 Weitere Abschnitte

Abschnitt	Skript	Kapitel
VERGLEICH	literaturvergleich.py	9
KALIBRIERUNG	kalibrierung.py	10
MONOLITH	monolith.py	11

6 Ergebnisse

6.1 Ablauf der Auswertung

`auswertung.py` führt für jede gewählte Messdatei folgende Schritte aus (Code: `taylorflow/pipeline.py`, Gleichungen: Kapitel 7):

```
Messdatei ──► Einlesen, Ausschlüsse, Kamerafilter                (einlesen.py)
   │
   └─► je Betriebspunkt (u_G, u_L):
         1  Messpunkte: je Kamera und Bin ein Wert
         2  Stoffdaten bei T und p am Auslass (thermo, Henry, Dampfdruck)
         3  Fits über z: Blasenlänge (mit Kamera-Offsets), Sluglänge,
          ↪  Geschwindigkeit
        4  Druckprofil: Rückwärtsintegration vom Auslass (Bretherton bzw. Kurimoto)
        5  Stoffbilanz: Blasenvolumen → absorbierte Stoffmenge → C_L, C*, Sättigung
         6  aktive Zone und kLa_log_mean
         7  Blasenlänge auf Einlassdruck umgerechnet
         8  Literaturmodelle
   │
   └─► Speichern: Profile, Betriebspunkt-Tabelle, Metadaten       (ergebnisse.py)
Danach: Plots nach config.PLOTS                                 (taylorflow/plots/)
```

6.2 Ergebnisdateien

Je Kanal entsteht `ergebnisse/<Kanal>/` mit:

Datei	Inhalt
`<Kanal>_betriebspunkte.csv`	eine Zeile je Betriebspunkt: Kennwerte, Fitgüte, Literaturmodelle
`<Kanal>_profile.csv`	eine Zeile je Gitterpunkt (1 mm) und Betriebspunkt
`<Kanal>_meta.json`	Programmversion, Zeitpunkt, Quelldatei
`plots/`	Plots nach `config.PLOTS` (Kapitel 4)

Format: Semikolon, Dezimalkomma, UTF-8; die Dateien lassen sich direkt in Excel öffnen. In Python liest man sie mit `taylorflow.ergebnisse.lade_betriebspunkte(ordner, kanal)` bzw. `lade_profile(ordner, kanal)`; diese Funktionen lesen Zahlen ohne Rundungsfehler. Eine vorhandene Datei gleichen Namens wird überschrieben.

Betriebspunkt-Tabelle

Spalte	Einheit	Bedeutung
`Config`	–	Name des Betriebspunkts, z. B. `Gas_0_134_Liq_0_056`
`GasVelocity_m_s`, `LiquidVelocity_m_s`	m/s	Leerrohrgeschwindigkeiten u_G, u_L

Spalte	Einheit	Bedeutung
kLa_log_mean_1_s	1/s	**Hauptergebnis:** logarithmisch gemittelter volumetrischer Stoffübergangskoeffizient über die aktive Zone (Kapitel 7.6); 0 bei zu kurzer aktiver Zone, leer bei kla_ungueltig
BubbleLength_mean_m, BubbleLength_start_m	m	gefittete Blasenlänge, Mittel über den Kanal bzw. am Einlass
BubbleLength_meas_mean_m	m	Mittel der gemessenen Blasenlängen
SlugLength_mean_m	m	gefittete Sluglänge, Mittel über den Kanal
Velocity_mean_m_s	m/s	gefittete Blasengeschwindigkeit, Mittel; Eingangsgröße der Literaturmodelle
Velocity_meas_mean_m_s	m/s	Mittel der gemessenen Geschwindigkeiten
slip_meas_over_theo	–	Median gemessene Geschwindigkeit / (u_G + u_L); üblich 1,05–1,30
n_spez_mol_s_m3	mol/(s·m³)	volumenspezifischer CO2-Molenstrom über den ganzen Kanal (Kapitel 7.7)
fit_model	–	verwendetes Blasenmodell; Normalfall Exp+Expansion v2+Offsets …
fit_r2, fit_chi2_red	–	Güte des Blasenfits; χ^2_red ≈ 1 ideal, > 3 auffällig, > 10 kritisch
fit_corr_max	–	größte Korrelation zweier Fitparameter; > 0,95 = schlecht bestimmbar
fit_n_bound_hits, fit_bound_hits	–	Parameter an einer Fitgrenze (Anzahl, Beschreibung)
fit_n_series, fit_n_offsets	–	Anzahl Kameras bzw. gefitteter Offsets
fit_offset_o2_mm, fit_offset_o3_mm	mm	Kamera-Offsets gegenüber Kamera 1
kLa_lit_*_1_s	1/s	Vorhersagen der Literaturmodelle (Kapitel 9)
d_h_m, Geometrie	m, –	Kanaldurchmesser bzw. Seitenlänge; rund oder eckig
D_CO2_m2_s	m²/s	Diffusionskoeffizient CO2 in Wasser bei T
T_sys_K, P_aus_Pa	K, Pa	Temperatur und Auslassdruck (Mittel des Betriebspunkts)
manual_exclude, manual_exclude_reason	–	True und Grund bei einem Eintrag unter kla_ungueltig

Profiltabelle

Spalte	Einheit	Bedeutung
Config_Name	–	Betriebspunkt
Position_z_m	m	Position im Kanal (Gitter mit dz_m)
L_bubble_fit_m	m	gefittete Blasenlänge (ohne Kamera-Offsets)

Spalte	Einheit	Bedeutung
`L_bubble_corrected_m`	m	Blasenlänge, umgerechnet auf den Einlassdruck; zeigt nur die Absorption
`L_slug_fit_m`, `U_fit_m_s`	m, m/s	gefittete Sluglänge und Blasengeschwindigkeit
`Pressure_Pa`	Pa	berechneter Absolutdruck
`Diffused_Gas_mol`	mol	bis z absorbierte CO2-Menge je Blase (monoton)
`C_L_bulk_mol_m3`	mol/m³	CO2-Konzentration in der Flüssigkeit
`C_star_lokal_mol_m3`	mol/m³	Gleichgewichtskonzentration C* = H · p_CO2
`Saturation_percent`	%	Sättigung C_L / C*
`C_star_lokal_raw_mol_m3`, `Saturation_percent_raw`	mol/m³, %	dieselben Größen vor der Monotonie- und Sättigungsbegrenzung
`aktive_zone`	–	1 = Gitterpunkt in der aktiven Zone (Kapitel 7.6), 0 = nicht
`fit_offset_o1_m` … `o3_m`	m	Kamera-Offsets (konstant je Betriebspunkt)
`cam1_z_min_m` … `cam3_z_max_m`	m	Bereich der Messpunkte je Kamera
`z_meas_m`, `L_b_meas_m`	m	Messpunkte (nur in den ersten Zeilen belegt, sonst leer)

6.3 Konsolenausgabe

Je Betriebspunkt meldet die Konsole:

- das Blasenmodell mit R^2 und χ^2_red
- die Kamera-Offsets
- bei runden Kanälen den verwendeten Bretherton-Faktor c
- Ein- und Auslassdruck
- kLa_log_mean und die Zahl der Gitterpunkte in der aktiven Zone

6.4 Warnungsprotokoll

`ergebnisse/Modell_Gueltigkeitswarnungen.txt` wird bei jedem Lauf neu geschrieben. Es sammelt, getrennt nach Kanal und Betriebspunkt:

Meldung	Bedeutung	Handlung
`Liu (2005): … außerhalb`	Betriebspunkt außerhalb des Bereichs der Schlupfkorrelation	meist keine, nur Hinweis
`Bretherton (1961): Ca_L … > 0.005`	Kapillarzahl über dem Gültigkeitsbereich des Bretherton-Druckmodells	Druckprofil mit Vorsicht bewerten

Meldung	Bedeutung	Handlung
`Kurimoto: …`	Ca, We, Re oder L_B/d außerhalb des Kurimoto-Bereichs	Druckprofil mit Vorsicht bewerten
`Filmdicke Modell: …`	Filmdicke außerhalb des Modellbereichs, Näherungswert verwendet	Betriebspunkt prüfen
`Blasen-Fit: chi2_red = … -> ERHEBLICH` bzw. `KRITISCH`	Fit beschreibt die Messung schlecht	Blasenlängen-Fit ansehen (Kapitel 4.2)
`Blasen-Fit: Parameter hoch korreliert`	A und λ (oder B) nicht unabhängig bestimmbar	kLa des Punkts kritisch prüfen
`Serien-Offset … nahe/am Bound`	Kameras stark gegeneinander versetzt	Kamerakalibrierung prüfen
`Serien-Offset … > 50% des Absorptionssignals`	Offset könnte einen Teil der Absorption aufnehmen	Plot prüfen, ggf. Kamera ausschließen
`Slip U_meas/U_TP > 1.5`	gemessene Geschwindigkeit unplausibel hoch	Sollwerte der Durchflussregler prüfen
`kla_ungueltig aktiv: …`	Eintrag aus `ausschluesse.yaml` angewendet	–
`kla_ungueltig: Betriebspunkt '…' nicht gefunden`	Name in `ausschluesse.yaml` passt zu keinem Betriebspunkt	Schreibweise prüfen (Spalte `Config`)

7 Methodik

Dieses Kapitel beschreibt die Modellkette so, wie sie im Code umgesetzt ist.

Bezeichnungen: z Position ab Einlass, L_B Blasenlänge, L_S Sluglänge, U Blasengeschwindigkeit, d Kanaldurchmesser bzw. Seitenlänge, δ Filmdicke, T Temperatur, p Druck, C_L Konzentration in der Flüssigkeit, C* Gleichgewichtskonzentration.

7.1 Annahmen

- **Isotherm:** Je Betriebspunkt werden T und p am Auslass gemittelt. Stoffdaten (ρ, μ, σ, H, D, p_v) gelten bei diesen Werten für den ganzen Kanal.
- **Gasphase:** Ideales Gas. Das Gas ist sofort mit Wasserdampf gesättigt; der trockene Gasdruck ist $p_{tr} = p - p_v$.
- **Absorption:** Nur CO2 wird absorbiert, der Inertanteil (N2) bleibt in der Blase.
- **Flüssigkeitsvolumen:** Das Flüssigkeitsvolumen einer Einheitszelle wird am Einlass bestimmt und bleibt konstant.
- **Durchmischung:** Der Slug ist ideal durchmischt (eine Konzentration C_L je Position).

7.2 Blasenlängen-Fit

Die gemessenen Blasenlängen aller Kameras werden mit dem Modell

$$L_B(z) = L_0 - A\left(1 - e^{-z/\lambda}\right) + B\,z \;+\; o_j$$

angepasst:

- A: absorptionsbedingte Verkürzung
- λ: Absorptionslänge
- B: Expansion durch den Druckabfall
- o_j: konstanter Versatz der Kamera j gegenüber Kamera 1 (o_1 = 0), der Kalibrierunterschiede zwischen den Kamerapositionen auffängt

Weitergerechnet wird mit der Kurve **ohne** Offsets.

Gewichtung:

$$\sigma = \sqrt{\sigma_{SEM}^2 + \sigma_{sys}^2}, \qquad \sigma_{SEM} = s/\sqrt{n} + 1\,\mu\text{m}, \quad \sigma_{sys} = 2\,\mu\text{m}.$$

Ablauf:

1. Gewichteter Fit.
2. Robuster Fit (soft-L1) mit einer Skalierung aus der MAD der Residuen des ersten Durchgangs.

Grenzen der Parameter:

- L_0 ∈ [0,1; 5] · max(L)
- A ≤ 2 · Spannweite(L)
- λ zwischen kleinstem Messpunktabstand und 10 L_Reaktor
- 0 ≤ B ≤ 2 max(L)/L_Reaktor
- |o_j| ≤ 0,2 · mittlere Blasenlänge

Ersatzmodelle: Gelingt der Fit nicht, folgen ein Modell ohne Offsets, dann $ae^{-bz} + c + dz$, zuletzt eine Gerade. Der verwendete Modellname steht in der Spalte `fit_model`.

Sluglänge $L_S = L_{UC} - L_B$ und Geschwindigkeit U werden linear über z angepasst. Gefittete Geschwindigkeiten über $1{,}5 \cdot U_{ref}$ werden verworfen und interpoliert; $U_{ref} = \max(\text{Median gemessen}, u_G + u_L)$.

7.3 Filmdicke und Blasengeometrie

Kapillarzahl $Ca = \mu U/\sigma$.

Kanal	Gültigkeit	Filmdicke
rund	Ca ≤ 0,005	Bretherton: $\delta = \frac{d}{2}\,1{,}34\,Ca^{2/3}$
rund	0,005 < Ca ≤ 1,5	Aussillous/Quéré: $\delta = \frac{d}{2}\,\frac{1{,}34\,Ca^{2/3}}{1 + 3{,}35\,Ca^{2/3}}$
quadratisch	–	Kreutzer/Thulasidas: $\delta/d = 0{,}357 - 0{,}25\,e^{-2{,}25\,Ca^{0{,}445}}$

Blase: Durchmesser bzw. Seitenlänge $d_B = d - 2\delta$. Sie besteht aus einem Zylinder (rund) bzw. Quader (quadratisch) der Länge $L_B - d_B$ und zwei Kappen mit zusammen $V_{caps} = \frac{\pi}{6} d_B^3$:

$$V_B = V_{caps} + A_B \max(0,\, L_B - d_B), \qquad A_B = \tfrac{\pi}{4} d_B^2 \text{ (rund)}, \quad A_B = d_B^2 \text{ (quadratisch)}.$$

Flüssigkeit je Einheitszelle (am Einlass), mit dem Kanalquerschnitt A_K:

$$V_L = L_S A_K + L_B (A_K - A_B) \; (+ V_{caps}/2 \text{ im runden Kanal}).$$

7.4 Druckprofil

Rückwärtsintegration vom gemessenen Auslassdruck mit Schritt dz = 1 mm: $p(z_i) = p(z_{i+1}) + (\mathrm{d}p/\mathrm{d}z)_i\,\Delta z$. Als Geschwindigkeit u_{TP} wird in beiden Modellen die gefittete Blasengeschwindigkeit $U(z)$ eingesetzt.

Runder Kanal (Bretherton 1961, nach Fries 2008):

$$f_{app} = \frac{16}{Re}\left[1 + c\,\frac{d}{L_S}\,Ca^{-1/3}\right], \qquad \Delta p_{UC} = 2 f_{app}\,\rho_L u_{TP}^2 \frac{L_S}{d}, \qquad \frac{\mathrm{d}p}{\mathrm{d}z} = \frac{\Delta p_{UC}}{L_S + L_B}.$$

c stammt aus der Kalibrier-Datenbank (Kapitel 10):

- lineare Interpolation zwischen den kalibrierten Betriebspunkten
- außerhalb davon das gespeicherte Ersatzmodell

- ohne verwendbaren Eintrag der Theoriewert c = 0,465

Quadratischer Kanal (Kurimoto et al. 2020, Gl. 27/28):

$$\Delta p_B = \frac{\sigma}{d}\,\frac{7{,}106\,Ca^{2/3}}{N}, \qquad \frac{\mathrm{d}p}{\mathrm{d}z} = \frac{L_S}{L_{UC}}\,\frac{\rho_L u^2}{2d}\,\frac{56{,}9}{Re}\left[1+\frac{d}{L_S}\,\frac{0{,}250\,Ca^{-1/3}}{N}\right],$$

mit $N = 1+0{,}744\,Ca^{2/3}-0{,}623\,We^{0{,}120}$. Es gibt keinen freien Faktor. Ca, Re und We werden mit der Blasengeschwindigkeit gebildet, wie bei Kurimoto vorgesehen.

7.5 Stoffbilanz

Gasmenge in der Blase an der Stelle z und am Einlass:

$$n(z) = \frac{p_{tr}(z)\,V_B(z)}{R\,T}, \qquad n_0 = n(0).$$

Davon sind $n_{CO_2,0} = y_{CO_2}\,n_0$ und $n_{inert} = (1-y_{CO_2})\,n_0$. Absorbiert bis z (begrenzt auf $[0,\, n_{CO_2,0}]$, danach monoton gemacht):

$$\Delta n(z) = n_0 - n(z), \qquad C_L(z) = \Delta n(z)/V_L.$$

Gleichgewicht mit dem lokalen CO2-Anteil im trockenen Gas:

$$y(z) = \frac{n_{CO_2,0}-\Delta n}{n_{inert}+n_{CO_2,0}-\Delta n}, \qquad C^*(z) = H(T)\,p_{tr}(z)\,y(z),$$

$$H(T) = H_{298}\,\exp\!\left[\frac{\Delta_{sol}H}{R}\left(\frac{1}{T}-\frac{1}{298{,}15}\right)\right]$$

mit $H_{298} = 3{,}36\cdot 10^{-4}\,\mathrm{mol/(m^3\,Pa)}$ und $\Delta_{sol}H/R = 2400\,\mathrm{K}$ für CO2 (Sander 2015).

C_L wird höchstens auf $0{,}999\,C^*$ gesetzt. Die Sättigung ist C_L/C^*.

7.6 Aktive Zone und kLa_log_mean

Aktive Zone. Ein Gitterpunkt gehört dazu, wenn die treibende Konzentrationsdifferenz $\Delta C = C^* - C_L$ positiv ist und größer als die doppelte Unsicherheit von C*:

$$\Delta C > 0 \quad \text{und} \quad \frac{u_{rel}(C^*)\,C^*}{\Delta C} \le 0{,}5, \qquad u_{rel}(C^*) = \sqrt{0{,}03^2+0{,}02^2} \approx 3{,}6\,\%.$$

Wird die Flüssigkeit nahezu gesättigt, ist ΔC nicht mehr sicher bestimmbar; dort endet die aktive Zone. Die Bedingung ist erfüllt, solange die Sättigung unter etwa 93 % bleibt; in den mitgelieferten Messreihen umfasst die aktive Zone daher meist nahezu den ganzen Kanal.

kLa_log_mean. Für eine Pfropfenströmung mit konstantem kLa gilt $\mathrm{d}C_L/\mathrm{d}t = k_L a\,(C^*-C_L)$. Integriert über die aktive Zone, vom ersten bis einschließlich zum letzten aktiven Punkt:

$$k_L a_{log} = \frac{1}{t_{res}}\,\ln\frac{\Delta C_{ein}}{\Delta C_{aus}}, \qquad t_{res} = \int_{z_{ein}}^{z_{aus}} \frac{\mathrm{d}z}{U(z)}$$

(Kashid 2011).

- kLa_log_mean nutzt ΔC an den Rändern der aktiven Zone und die Verweilzeit. Es ist damit unempfindlich gegen Rauschen in den Zwischenwerten.
- Es wird 0 bei weniger als 10 aktiven Gitterpunkten oder wenn ΔC in der aktiven Zone nicht abnimmt.
- Es bleibt leer, wenn der Betriebspunkt unter `kla_ungueltig` steht.

7.7 Weitere Kenngrößen

Volumenspezifischer Molenstrom über den ganzen Kanal (Bezug wie beim Monolithen):

$$\dot{n}_{spez} = \frac{\Delta n(z_{Ende})\, f_B}{A_K\, z_{Ende}}, \qquad f_B = \frac{U(0)}{L_B(0) + L_S(0)}$$

(f_B ist die Blasenfrequenz).

Druckkorrigierte Blasenlänge: Das Blasenvolumen wird auf den Einlassdruck umgerechnet, $V_B\, p_{tr}(z)/p_{tr}(0)$, und zurück in eine Länge. Ohne Absorption wäre die Kurve konstant.

Absorptionslänge z95 (Kennzahlen-Übersicht): Position, an der Δn 95 % seines Werts am Kanalende erreicht, bezogen auf die Kanallänge.

Schlupf: Median der gemessenen Blasengeschwindigkeit / (u_G + u_L).

7.8 Messunsicherheit

Eine Fehlerfortpflanzung für den Stoffübergang entlang z (Unsicherheiten von Stoffstrom, Flüssigkeitsvolumen und ΔC) ist als Funktion `relative_unsicherheit` in `taylorflow/unsicherheit.py` enthalten. Die Auswertung nutzt sie nicht; sie steht für eigene Betrachtungen zur Verfügung.

7.9 Literatur

- Aussillous, P., Quéré, D. (2000). Quick deposition of a fluid on the wall of a tube. *Phys. Fluids* 12, 2367–2371.
- Bretherton, F. P. (1961). The motion of long bubbles in tubes. *J. Fluid Mech.* 10, 166–188.
- Fries, D. M. et al. (2008). Gas–liquid two-phase flow in meandering microchannels. *Chem. Eng. J.* 135S, S37–S45.
- Kashid, M. N. et al. (2011). Gas–liquid and liquid–liquid mass transfer in microstructured reactors. *Chem. Eng. Sci.* 66, 3876–3897.
- Kreutzer, M. T. (2003). *Hydrodynamics of Taylor flow in capillaries and monolith reactors.* Diss., TU Delft.
- Kurimoto, R. et al. (2020). Pressure drop of Taylor flow in square microchannels. *Chem. Eng. Sci.*
- Liu, H. et al. (2005). Hydrodynamics of Taylor flow in vertical capillaries. *Ind. Eng. Chem. Res.* 44, 4884–4897.
- Sander, R. (2015). Compilation of Henry's law constants for water as solvent. *Atmos. Chem. Phys.* 15, 4399–4981.

- Versteeg, G. F., van Swaaij, W. P. M. (1988). Solubility and diffusivity of acid gases in aqueous alkanolamine solutions. *J. Chem. Eng. Data* 33, 29–34.

Quellen der Literaturmodelle: Kapitel 9.

8 Grenzen

8.1 Gültigkeitsbereiche der Modelle

Modell	Bereich	Verhalten außerhalb
Bretherton-Druckverlust (rund)	Ca ≤ 0,005	Warnung im Protokoll, Rechnung läuft weiter
Kurimoto-Druckverlust (quadratisch)	0,00159 ≤ Ca ≤ 0,0989; 0,0817 ≤ We ≤ 25,4; 8,33 ≤ Re ≤ 791; L_B/d ≥ ca. 2,3	Warnung, Rechnung läuft weiter
Filmdicke rund	Ca ≤ 1,5	Näherung δ = 0,05 d mit Warnung
Schlupf Liu (2005)	0,008–0,7 m/s (Gas), 0,008–0,5 m/s (Flüssigkeit), d 0,9–3 mm	Warnung (nur Hinweis, keine Rechengröße)
Henry-Konstante	CO2, O2, N2, Luft in Wasser	Abbruch mit Meldung
CO2-Diffusion	Wasser	–
Literaturmodelle	siehe Kapitel 9.2; die mitgelieferten Kanäle liegen bei fast allen Modellen außerhalb des Originalbereichs	–

Im mitgelieferten Glaskanal unterschreiten einige Betriebspunkte bei kleinem u_G das Kurimoto-Kriterium L_B/d leicht.

8.2 Bekannte Einschränkungen

1. **Isotherm, Stoffdaten am Auslass:**
   - T und p am Auslass gelten für den ganzen Kanal.
   - Bei Temperaturänderungen über ca. 5 K entlang des Kanals wäre eine abschnittsweise Auswertung nötig.
2. **Konstantes Flüssigkeitsvolumen:**
   - V_L wird am Einlass bestimmt.
   - Änderungen der Sluglänge entlang z gehen nicht in C_L ein.
3. **Kappen und Zwickel im quadratischen Kanal:**
   - Das Zwickelvolumen in den Ecken wird nur im runden Kanal (als halbes Kappenvolumen) zum Flüssigkeitsvolumen gezählt.
   - Die Kappen werden in beiden Geometrien als Halbellipsoide genähert.
   - Eine Würfel- statt Ellipsoidnäherung der Kappen ändert im Glaskanal einzelne kLa-Werte um bis zu 26 % (Median 1,8 %).

4. **Druckmodell der runden Kanäle:**
    - Kein veröffentlichtes Druckmodell (Bretherton, Kreutzer 2005, Warnier 2010) gibt alle drei mitgelieferten runden Kanäle ohne kanalspezifische Anpassung mit weniger als 70 % mittlerer Abweichung wieder.
    - Für PVC sind deshalb kalibrierte c-Werte hinterlegt (Kapitel 10).
    - Der Kanal PVC 1500/1150 ist als nicht verwendbar markiert (c zwischen −7,6 und +16); dort gilt c = 0,465.
5. **Eckige und runde Kanäle im Vergleich:**
    - Im Glaskanal (eckig) folgen die Messwerte dem Trend der Literaturmodelle, besonders Yue 2007; die Absolutwerte weichen teils um ein Mehrfaches ab.
    - Die runden Kanäle zeigen kaum einen Trend. Wahrscheinliche Ursachen sind das Druckmodell und Materialeffekte.
6. **Aktive Zone als ein Bereich:**
    - kLa_log_mean nutzt den ersten und den letzten aktiven Punkt.
    - Wäre die aktive Zone unterbrochen, zählten die Abschnitte dazwischen zur Verweilzeit mit. In den mitgelieferten Messreihen kommt das nicht vor.
7. **Literaturmodelle bei 25 °C:**
    - Ca, Re und Sc der Literaturmodelle werden mit festen Stoffwerten für etwa 25 °C berechnet.
    - Die übrige Auswertung nutzt die gemessene Temperatur.
    - Bei 20–25 °C beträgt der Unterschied wenige Prozent.
8. **Zwischenspeicher der Stoffdaten:**
    - Stoffdaten werden mit auf 0,1 K gerundeter Temperatur zwischengespeichert.
    - Betriebspunkte mit fast gleicher Temperatur erhalten identische Werte. Das Ergebnis eines Betriebspunkts hängt damit geringfügig davon ab, welche Punkte im selben Lauf vorher gerechnet wurden: Wird ein Betriebspunkt zusätzlich ausgewertet oder ausgeschlossen, ändern sich andere kLa_log_mean-Werte um höchstens etwa 0,002 %.
    - Die Referenzergebnisse gelten für die Auswertung aller mitgelieferten Kanäle in alphabetischer Reihenfolge.
9. **3D-Flächen:**
    - Die kubische Interpolation kann bei wenigen, unregelmäßig verteilten Punkten über- oder unterschwingen.
    - Für quantitative Aussagen gelten die Messpunkte.
10. **Mehrere Aufnahmen je Bin:** Enthält eine Messdatei mehrere Zeilen für dieselbe Kamera und dasselbe Bin, wird die kleinste Blasenlänge verwendet (Kapitel 2.3).

8.3 Empfindlichkeit gegenüber den Einstellungen

Alle festen Schwellwerte wurden mit den mitgelieferten Messreihen variiert (5 Kanäle, 55 Betriebspunkte). Wirkung auf kLa_log_mean:

Größe	Variation	Effekt
Startwert λ_0 des Fits	0,01–0,2 m	keiner (identische Ergebnisse)
Ortsschritt dz	0,25–2 mm	im Prozentbereich, unter der Messunsicherheit
Sättigungsgrenze	0,99–0,999	keiner
Grenze aktive Zone u_rel(ΔC)	0,3–0,7	keiner
Deckel für U_fit	–	nie aktiv
Offset-Grenze	–	nie erreicht
Ersatzmodell der Kalibrierung	Linear statt Polynomial	≤ 3,6 % in einem Betriebspunkt
Filmdickenmodell	δ/d 0,03–0,10, Aussillous statt Bretherton	keiner (Ersatzwert nie aktiv)

Die Ergebnisse hängen damit praktisch nicht von den numerischen Einstellungen ab. Maßgeblich sind die Messdaten, das Druckmodell und die Ausschlussregeln.

8.4 Offene Punkte

- **Druckmodell runde Kanäle:** Eine physikalisch begründete Alternative zur Kalibrierung fehlt (8.2 Nr. 4).
- **Kalibrierung:** Die Ersatzmodelle werden mit nur 5–8 Punkten angepasst (Kapitel 10.5).
- **Rechteckige, nicht quadratische Kanäle:** Geometrie, Druck- und Filmmodell müssten erweitert werden.
- **Andere Flüssigkeiten:** Henry-Konstante und Diffusionskoeffizient müssten ergänzt werden (Kapitel 12.3).

9 Literaturvergleich und eigene Modelle

`python literaturvergleich.py` vergleicht eine gemessene Kenngröße mit Literaturmodellen und passt auf Wunsch ein eigenes Modell an.

9.1 Ablauf

1. `auswertung.py` muss für die gewünschten Kanäle gelaufen sein.
2. In `config.py`, Abschnitt `VERGLEICH`, festlegen:

Eintrag	Standard	Bedeutung
`kanaele`	`None`	`None` = alle Kanäle mit Ergebnissen; sonst Liste von Kanalnamen
`zielgroesse`	`"kLa_log_mean_1_s"`	beliebige Zahlenspalte der Betriebspunkt-Tabelle
`groesse_beschriftung`	`r"$k_La_{\mathrm{log}}$ [1/s]"`	Achsenbeschriftung der Zielgröße
`modelle`	alle Literaturmodelle	Liste der Modelle für Tabelle und Paritätsdiagramme
`band_prozent`	15	Breite des Bands ± um die Diagonale [%]
`logarithmisch`	`True`	logarithmische Achsen
`eigener_fit`	Block	eigenes Modell anpassen (9.3); `None` = kein Fit

3. Ausführen. Ausgaben in `ergebnisse/vergleich/`:
   - `vergleich_<Zielgröße>.csv`: Statistik je Modell, sortiert nach MAPE
   - `paritaet_<Modell>.png`: Paritätsdiagramm je Modell, Kanäle farbig
   - `eigenes_modell_<Name>.png`: Parität des angepassten eigenen Modells

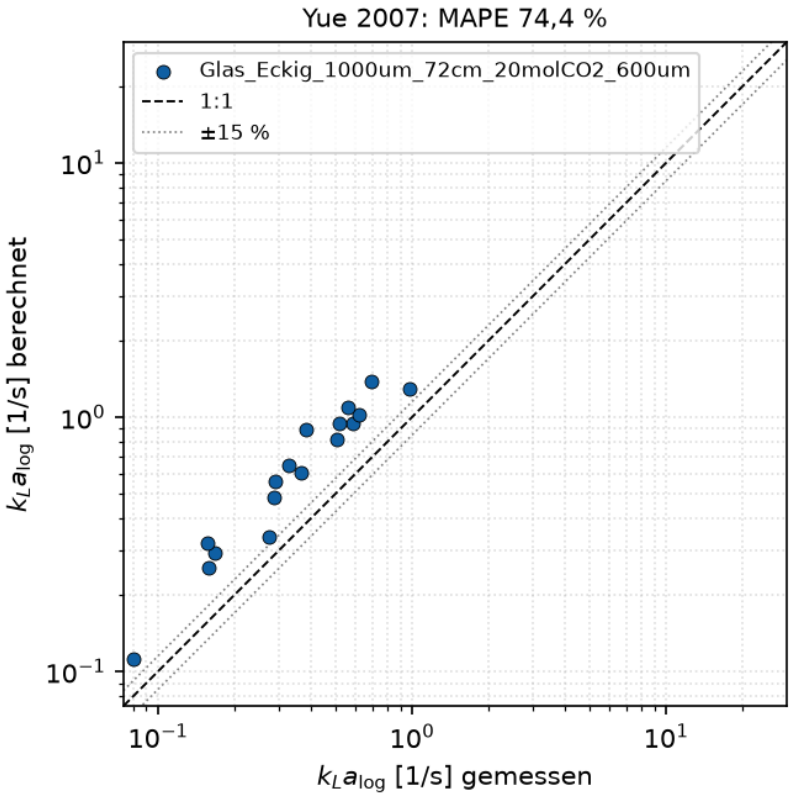


Abbildung 6: Paritätsdiagramm Yue 2007 für den Glaskanal

Paritätsdiagramm: gemessene gegen berechnete Werte, Kanäle mit eigener Farbe und Form. Die gestrichelte Diagonale ist die perfekte Übereinstimmung, die gepunkteten Linien begrenzen die Abweichung ±`band_prozent`. Der Titel nennt den MAPE. Kanäle, für die das Modell nicht gilt (hier die runden Kanäle bei Yue 2007), fehlen. Größe und Sprache folgen `STIL` (`groesse_paritaet_cm`, `sprache`, Kapitel 4.10).

Betriebspunkte mit `manual_exclude = True` werden nicht verwendet.

Statistik in `vergleich_<Zielgröße>.csv`:

Spalte	Bedeutung
`n`	Anzahl Punkte mit gültigem Mess- und Modellwert
`MAPE_%`	mittlere absolute relative Abweichung
`Bias_%`	Median der relativen Abweichung; positiv = Modell zu hoch
`RMSE`	Wurzel der mittleren quadratischen Abweichung (Einheit der Zielgröße)
`im_Band_%`	Anteil der Punkte innerhalb ±`band_prozent`

9.2 Literaturmodelle

Alle Modelle stehen in `taylorflow/literatur.py` mit Formel und Originalbereich. Ihre Vorhersagen sind außerdem als Spalten `kLa_lit_*_1_s` in jeder Betriebspunkt-Tabelle enthalten.

Name	Quelle	Kanal	Formel (Kurzform)	Original
van_baten_voll	van Baten, Krishna 2004	beide	Kappe + Film, Penetrationstheorie	rund, d 1,5–3 mm, L_UC 15–50 mm
van_baten_kappe, van_baten_film	dto.	beide	Anteile von Kappe bzw. Film	
vandu	Vandu et al. 2005	beide	4,5 √(D u_B / L_UC) / d	d 1–3 mm
shao	Shao et al. 2010	beide	3,0 √(D u_B / L_UC) / d	CFD, CO2/NaOH
liu_wang	Liu, Wang 2011	rund	0,12 Li^0,44 Pe^0,54 D/d²	d 1–3 mm, L_S 8–28 mm
heiszwolf	Heiszwolf et al. 2001	beide	0,133 u^1,2 / √L_S · (D/D_ref)^0,5	Monolith, quadratisch
yao	Yao et al. 2023	beide	van Baten mit Ca-Exponenten	0,3 × 0,6 mm
yue_2007	Yue et al. 2007	eckig	0,084 Re_G^0,213 Re_L^0,937 Sc^0,5 D/d²	0,5 × 1 mm, CO2/Wasser
yue_2009	Yue et al. 2009	eckig	(2/d) √(D u_B/d) (L_B/L_UC)^0,3	d 0,2–0,4 mm

Eingangsgrößen:

- gemittelte gefittete Blasengeschwindigkeit u_B, Blasen- und Sluglänge sowie d und D(T) des Betriebspunkts
- Yue 2007 nutzt die Leerrohrgeschwindigkeiten
- Stoffwerte für Ca, Re und Sc sind fest (Wasser/CO2, ca. 25 °C)

Die mitgelieferten Kanäle liegen bei fast allen Modellen außerhalb des Originalbereichs (größere Kanäle, längere Einheitszellen); die Abweichungen sind entsprechend groß. Nur Yue 2007 folgt im Glaskanal dem Trend (Steigung im log-log-Diagramm 0,92, $R^2 = 0{,}92$), mit rund 70 % systematischer Überschätzung.

9.3 Eigenes Modell anpassen

Der Block `eigener_fit` in `config.VERGLEICH` passt einen Potenzansatz an:

$$y = a \cdot x_1^{b_1} \cdot x_2^{b_2} \cdots$$

- y ist die Zielgröße.
- x_k sind beliebige Zahlenspalten der Betriebspunkt-Tabelle, z. B. `GasVelocity_m_s`, `LiquidVelocity_m_s`, `BubbleLength_mean_m`, `SlugLength_mean_m`, `Velocity_mean_m_s`, `d_h_m`.
- Angepasst wird mit kleinsten Quadraten auf log y. Das minimiert relative Abweichungen.
- Ausgegeben werden die Formel, n, MAPE und R^2 im logarithmischen Raum.

```
VERGLEICH = dict(
    ...
    eigener_fit=dict(
        name="pvc_kla",                    # Name zum späteren Aufruf
        titel="PVC: kLa = a uG^b uL^c",
        kanaele=["PVC_Rund_1000um_72cm_20molCO2_600um",
                 "PVC_Rund_1000um_72cm_20molCO2_800um"],
```

```
        variablen=["GasVelocity_m_s", "LiquidVelocity_m_s"],
        speichern=True,
    ),
)
```

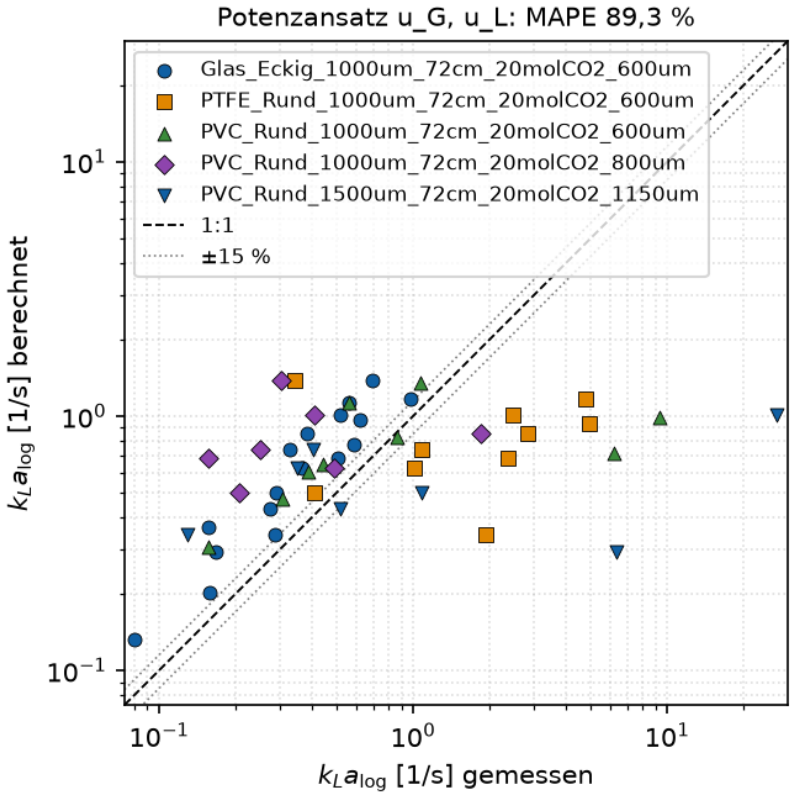


Abbildung 7: Potenzansatz in u_G und u_L, angepasst an alle Kanäle

Mit `speichern=True` wird das Modell in `config/eigene_modelle.yaml` abgelegt; ein gleichnamiger Eintrag wird überschrieben. Danach steht es unter seinem Namen überall wie ein Literaturmodell zur Verfügung:

- in `VERGLEICH["modelle"]`
- im Plot `modell_flaechen` (Kapitel 4.7)
- in `MONOLITH["vergleich_modell"]` (Kapitel 11)

Aufbau eines gespeicherten Eintrags (Zahlenwerte nur zur Veranschaulichung):

```yaml
pvc_kla:
  titel: 'PVC: kLa = a uG^b uL^c'
  zielgroesse: kLa_log_mean_1_s
  variablen: [GasVelocity_m_s, LiquidVelocity_m_s]
  koeffizient: 12.3
  exponenten: [0.61, 0.95]
  form: potenz
  datengrundlage: [PVC_Rund_1000um_72cm_20molCO2_600um, PVC_Rund_1000um_72cm_
↪   20molCO2_800um]
  n_punkte: 16
  mape_prozent: 24.1
  r2_log: 0.81
  erstellt: '2026-10-14'
```

Die Datei darf auch von Hand bearbeitet werden, z. B. um eine Literaturkorrelation in Potenzform mit eigenen Koeffizienten einzutragen.

Andere Zielgrößen: Jede Zahlenspalte ist möglich, z. B. `n_spez_mol_s_m3` oder `slip_meas_over_theo`. Die Literaturmodelle beziehen sich alle auf kLa; für andere Größen sind nur eigene Modelle sinnvoll.

9.4 Aus Python

```
import config
from taylorflow import plots
from taylorflow.modellvergleich import (lade_betriebspunkte_mehrerer_kanaele, fit_
 ↪  potenzansatz,
                                        vergleichstabelle, vorhersage)

daten = lade_betriebspunkte_mehrerer_kanaele(config.ERGEBNIS_ORDNER)
glas = daten[daten["Kanal"].str.startswith("Glas")]

# Anderer Ansatz: kLa über Blasengeschwindigkeit und Einheitszellenlänge
glas = glas.assign(L_UC_m=glas["BubbleLength_mean_m"] + glas["SlugLength_mean_m"])
m = fit_potenzansatz(glas, "kLa_log_mean_1_s", ["Velocity_mean_m_s", "L_UC_m"],
 ↪  name="glas_ub_luc")
print(m.formel(), m.mape_prozent)

print(vergleichstabelle(glas, "kLa_log_mean_1_s", ["yue_2007", "vandu"]))

fig = plots.paritaet(glas["kLa_log_mean_1_s"], {"Yue 2007": vorhersage(glas, "yue_
 ↪  2007")},
                     plots.PlotStil(), r"$k_La$ [1/s]", band_prozent=30,
 ↪  logarithmisch=True)
plots.speichere(fig, "abbildungen", "paritaet_yue", plots.PlotStil())
```

Ein Modell, das abgeleitete Spalten wie hier `L_UC_m` nutzt, braucht diese Spalte auch beim späteren Aufruf. Für die dauerhafte Nutzung über `config.py` sind daher die vorhandenen Spalten der Betriebspunkt-Tabelle vorzuziehen.

9.5 Neues Literaturmodell als Formel ergänzen

Für Modelle, die kein Potenzansatz sind, wird in `taylorflow/literatur.py` eine Funktion geschrieben und in `LITERATURMODELLE` eingetragen (Kapitel 12.3). Danach erscheint das Modell automatisch als Spalte in den Betriebspunkt-Tabellen (nach erneuter Auswertung), im Vergleich und als Fläche.

10 Druckkalibrierung

> **Status:** Das Verfahren ist vollständig enthalten, aber noch nicht abschließend validiert und wird derzeit nicht routinemäßig genutzt. Die Auswertung liest nur die vorhandene Datenbank `data/calibrated_c_database.json`. `kalibrierung.py` verändert diese Datei nur, wenn es ausdrücklich eingestellt ist.

10.1 Wozu

Im runden Kanal wird der Druckverlust mit dem Bretherton-Modell berechnet (Kapitel 7.4). Es enthält einen Faktor c mit dem Theoriewert 0,465. Mit diesem Wert passt das berechnete Druckprofil nicht zu allen Kanälen, und kein anderes veröffentlichtes Modell passt besser (Kapitel 8.2). c wird deshalb je Kanal aus Messungen ohne CO2 bestimmt; dort ändert sich die Blasenlänge nur durch den Druck.

Für quadratische Kanäle (Kurimoto-Modell) wird nicht kalibriert.

10.2 Verfahren

Für jeden Betriebspunkt einer Datei `Auswertung_Druck_…_0molCO2_….csv`:

1. **Fits:** Blasenlänge (gewichtet), Sluglänge und Geschwindigkeit linear über z.
2. **Realer Druckverlust** aus der Blasenexpansion (ideales Gas, Volumen proportional zur Länge):

$$p_{tr,ein} = p_{tr,aus}\,\frac{L_B(z_{Ende})}{L_B(0)}, \qquad \Delta p_{real} = p_{tr,ein} + p_v - p_{aus}$$

3. **Zerlegung des Modells:** Das Bretherton-Modell ist linear in c:
   - $\mathrm{d}p/\mathrm{d}z = \text{Basis} + c \cdot \text{Zusatz}$
   - Basis = Modell mit c = 0
   - Zusatz = Modell(c = 1) − Modell(c = 0)
   - Beide Anteile werden über den Kanal integriert.
4. **Ergebnis:** $c = (\Delta p_{real} - \Delta p_{Basis})/\Delta p_{Zusatz}$

Über alle Betriebspunkte werden drei **Ersatzmodelle** für c(u_G, u_L) angepasst:

Modell	Form
Linear	$a\,u_G + b\,u_L + c_0$
Polynomial	$a\,u_G + b\,u_L + c_1\,u_G u_L + c_0$
Power-Law	$a\,u_G^{b}\,u_L^{c_1}$

Ausgegeben werden je Modell R^2 und AIC (kleiner = besser; bestraft zusätzliche Parameter).

10.3 Nutzung in der Auswertung

Für einen runden Kanal wird der Eintrag mit dem Kalibrierschlüssel gesucht, z. B. `PVC_circular_1000um_600um` (Kapitel 2.2). Dann gilt:

Lage	verwendetes c
innerhalb der kalibrierten Betriebspunkte	lineare Interpolation
außerhalb	gespeichertes Ersatzmodell (`fallback_model`)
kein Eintrag, `"USE": false` oder weniger als 4 Punkte	Theoriewert 0,465 (Meldung in der Konsole)

Das Ergebnis ist mindestens 0,001.

Inhalt der Datenbank:

Schlüssel	Punkte	c	Ersatzmodell	verwendet
`PVC_circular_1000um_600um`	5	0,31–1,33	Power-Law (R^2 0,91)	ja
`PVC_circular_1000um_800um`	8	0,05–0,70	Polynomial (R^2 0,58)	ja
`PVC_circular_1500um_1150um`	8	−7,6 bis +16	Polynomial	nein (`USE: false`)
`Glas_rectangular_1000um_600um`	8	–	–	nein; eckig, Kurimoto
PTFE	–	–	–	kein Eintrag, c = 0,465

Ein Eintrag lässt sich von Hand abschalten:

```
"PVC_circular_1500um_1150um": {
    "USE": false,
    "NOTE": "c_calib zwischen -7.6 und +16, vermutlich Messfehler",
    ...
}
```

10.4 Neu kalibrieren

1. Mit der Videoauswertung die Druckmessung auswerten; die Datei `Auswertung_Druck_….csv` entsteht in `output/Output_PressureCalibration/`. Material, Form, D und Nadel müssen zur Stoffübergangsmessung passen.
2. In `config.py`, Abschnitt `KALIBRIERUNG`:

Eintrag	Standard	Bedeutung
`messdateien`	`"dialog"`	`"dialog"` = Auswahlfenster, `"alle"` = alle `Auswertung_Druck_*.csv` in `ROHDATEN_ORDNER`, oder Liste von Pfaden

Eintrag	Standard	Bedeutung
`dialog_ordner`	`ROHDATEN_ORDNER`	Startordner des Auswahlfensters, z. B. `Output_PressureCalibration` der Videoauswertung
`ersatzmodell`	`"auto"`	`"auto"` (größtes R^2) oder `"Linear"`, `"Polynomial"`, `"Power-Law"`
`datenbank_schreiben`	`False`	zuerst `False` lassen

3. `python kalibrierung.py` ausführen und die Datei(en) auswählen.
4. Ergebnisse in `ergebnisse/kalibrierung/` prüfen:
   - `<Schlüssel>_c_werte.csv`: c je Betriebspunkt
   - `<Schlüssel>_c_flaeche.png`: c über u_G und u_L

   Plausibel sind Werte in der Größenordnung des Theoriewerts (0,1–2). Negative oder stark streuende Werte deuten auf ein Problem in der Druckmessung oder den Fits hin.
5. Erst dann `datenbank_schreiben = True` setzen und erneut ausführen. Vorhandene Felder `USE` und `NOTE` bleiben erhalten. Danach `auswertung.py` für den Kanal erneut laufen lassen.

Eine Neukalibrierung der mitgelieferten Druckdaten reproduziert die gespeicherten c-Werte auf etwa 10^{-4} genau und wählt dieselben Ersatzmodelle.

10.5 Offene Fragen

- **Annahme Volumen proportional zur Länge:** Kappen werden vernachlässigt, die Filmdicke gilt als konstant. Bei kurzen Blasen ist die Annahme grob.
- **Anpassung mit wenigen Punkten:** Fünf bis acht Punkte je Kanal sind wenig, um drei bis vier Parameter anzupassen. `auto` wählt nach R^2 und bevorzugt damit das Polynom; nach AIC wäre für PVC 1000/800 das lineare Modell zu wählen. Für Betriebspunkte innerhalb der kalibrierten Punkte spielt das keine Rolle (Interpolation).
- **PVC 1500/1150:** Die stark streuenden c-Werte sind nicht erklärt.
- **Messabweichung der Blasenlänge:** Sie geht in der Kalibrierung nur über die Standardabweichung ein, nicht über Standardfehler und systematischen Anteil wie im Blasenlängen-Fit der Stoffübergangsmessung.

11 Monolith

`python monolith.py` wertet Messungen an einem Monolithen über eine Gasbilanz aus und vergleicht sie mit einem Einzelkanal.

11.1 Messdaten und Einstellungen

Die Messdatei (Standard `data/raw/Monolith_Messungen.csv`) enthält eine Zeile je Betriebspunkt, Semikolon-getrennt und mit Dezimalpunkt:

Spalte	Einheit	Bedeutung
`V_L_L_h`	L/h	Flüssigkeitsvolumenstrom
`V_G_in_L_h`, `V_G_out_L_h`	L/h	Gasvolumenstrom am Ein- und Austritt
`y_CO2_in`, `y_CO2_out`	%	CO2-Anteil am Ein- und Austritt (Analysator)
`T_in_C`, `T_out_C`	°C	Temperatur
`p_in_Pa`, `p_out_Pa`	Pa	Absolutdruck

Einstellungen in `config.py`, Abschnitt `MONOLITH`:

Eintrag	Standard	Bedeutung
`datei`	`data/raw/Monolith_Messungen.csv`	Messdatei
`n_kanaele`	100	Anzahl Kanäle
`d_kanal_m`	1e-3	Seitenlänge der quadratischen Kanäle [m]
`laenge_m`	1,0	Länge [m]
`henry_ref_mol_m3_pa`	3,3e-4	Henry-Konstante bei 298,15 K [mol/(m³·Pa)]
`henry_dlnh_dt_k`	2400	Temperaturabhängigkeit d(ln H)/d(1/T) [K]
`vergleich_kanal`	Glaskanal	Kanalname für den Vergleich; `None` = kein Vergleich
`vergleich_modell`	`"yue_2007"`	zusätzliche Modellfläche; `None` = keine

11.2 Rechnung

Alle Kanäle werden gemeinsam betrachtet. Gesamtquerschnitt $A = N d^2$, Volumen $V = A\,L$.

Leerrohrgeschwindigkeiten:

$$u_G = \dot{V}_{G,ein}/A, \qquad u_L = \dot{V}_L/A$$

Absorbierter Molenstrom aus der Gasbilanz, mit $c_G = p\,y/(RT)$:

$$\dot{n} = \dot{V}_{G,ein}\,c_{G,ein} - \dot{V}_{G,aus}\,c_{G,aus}, \qquad \dot{n}_{spez} = \dot{n}/V$$

Gleichgewicht, Verweilzeit und Austrittskonzentration:

$$C^* = H(T)\,p\,y, \qquad t = L/u_L, \qquad C_{L,aus} = \dot{n}\,t/(\dot{V}_L t)$$

kLa mit logarithmischer Konzentrationsdifferenz ($\Delta C_{ein} = C^*_{ein}$, $\Delta C_{aus} = C^*_{aus} - C_{L,aus}$):

$$k_L a = \frac{\dot{n}}{V_L\,\Delta C_{log}}, \qquad \Delta C_{log} = \frac{\Delta C_{ein} - \Delta C_{aus}}{\ln(\Delta C_{ein}/\Delta C_{aus})}$$

Die Henry-Konstante folgt van't Hoff mit den Werten aus `MONOLITH`; sie weichen leicht von denen der Kanalauswertung ab (3,36·10⁻⁴ mol/(m³·Pa), Kapitel 7.5).

Gesättigte Betriebspunkte: Ist ΔC_aus ≤ 0, ist die Flüssigkeit am Austritt gesättigt und kLa nicht bestimmbar (leer). In den mitgelieferten Daten betrifft das 14 von 25 Betriebspunkten.

11.3 Ausgaben

In `ergebnisse/monolith/`:

Datei	Inhalt
`monolith_kennwerte.csv`	u_G, u_L, kLa, ṅ_spez, Sättigungskennung je Betriebspunkt
`kla_monolith.png`, `nspez_monolith.png`	3D-Flächen des Monolithen
`kla_kanal.png`, `nspez_kanal.png`	dieselben Größen für `vergleich_kanal`
`kla_modell.png`	Modellfläche (`vergleich_modell`)
`paritaet_monolith_kanal.png`	kLa des Kanals gegen kLa des Monolithen

Alle 3D-Flächen eines Laufs haben dieselben Achsbereiche und sind direkt vergleichbar. Sprache, Format und Größen folgen `STIL` (Kapitel 4.10).

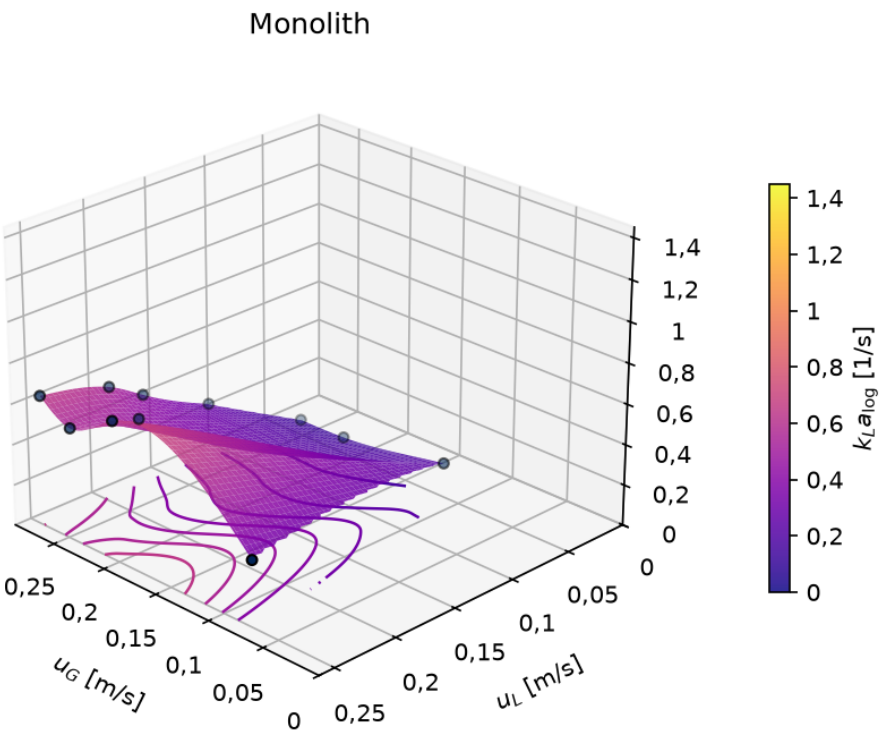


Abbildung 8: kLa des Monolithen

Paritätsdiagramm:

- Die Kanalwerte werden auf die Betriebspunkte des Monolithen interpoliert.
- Für gesättigte Monolith-Punkte wird der Monolith-Wert aus der eigenen Fläche interpoliert und gesondert markiert.

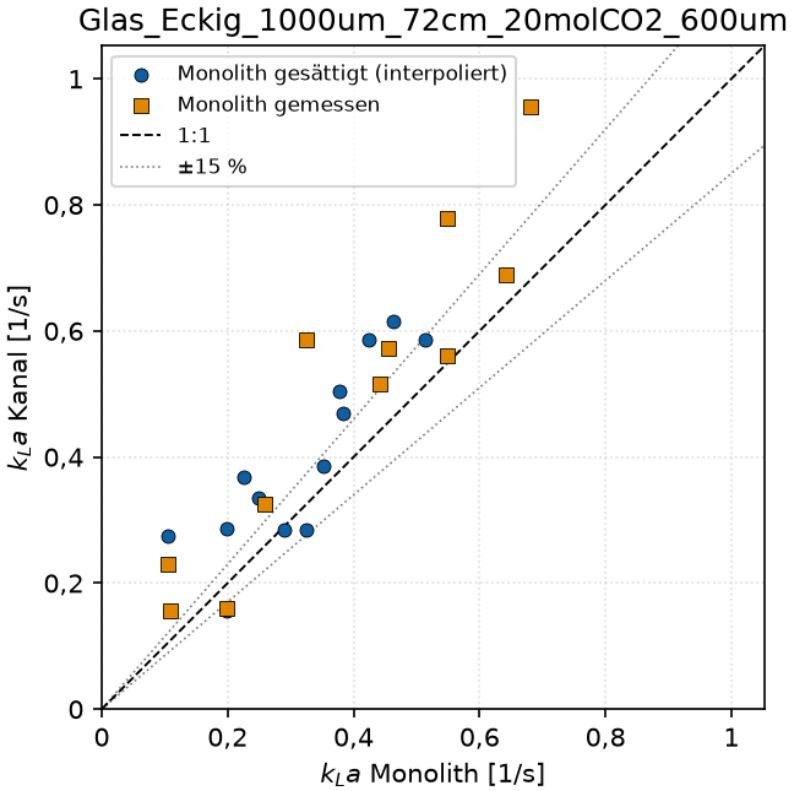


Abbildung 9: Parität Monolith gegen Glaskanal

11.4 Einordnung

ṅ_spez des Kanals ist absichtlich auf das gesamte Kanalvolumen bezogen, wie beim Monolithen (Kapitel 7.7). kLa des Kanals ist dagegen das Log-Mean über die aktive Zone (Kapitel 7.6), kLa des Monolithen ein Log-Mean über den ganzen Reaktor. Bei frühzeitiger Sättigung sind die beiden kLa-Werte deshalb nur eingeschränkt vergleichbar.

12 Programmaufbau und Erweiterung

12.1 Aufbau

```
config.py              alle Einstellungen
auswertung.py          Auswertung und Plots
abbildung.py           einzelne Abbildung
literaturvergleich.py  Modellvergleich
kalibrierung.py        Druckkalibrierung
monolith.py            Monolith
taylorflow/
  parameter.py         Zahlenwerte der Auswertung (Dataclass AuswerteParameter)
  einlesen.py          Dateiauswahl, Dateinamen, Messdaten, Ausschlüsse,
   ↪  Kamerafilter
  pipeline.py          Auswertung je Kanal und Betriebspunkt (Schritte 1–8, Kapitel
   ↪  6.1)
  fits.py              Blasenmodell mit Kamera-Offsets, lineare Fits
  druckverlust.py      Bretherton, Kurimoto, Kalibrier-Datenbank
  hydrodynamik.py      Kapillarzahl, Filmdicke
  geometrie.py         Blasenvolumen und -querschnitt
  stoffdaten.py        thermo-Anbindung, Henry, Dampfdruck, Diffusion
  kla.py               aktive Zone, kLa_log_mean
  unsicherheit.py      Fehlerfortpflanzung (wird von der Auswertung nicht
   ↪  aufgerufen)
  literatur.py         Literaturmodelle und Registry
  modellvergleich.py   Vorhersagen, Potenzansatz-Fit, Statistik, eigene Modelle
  ergebnisse.py        Speichern und Laden der Ergebnisdateien
  protokoll.py         Warnungsprotokoll
  monolith.py          Gasbilanz des Monolithen
  kalibrierung.py      Druckkalibrierung
  plots/
    stil.py            PlotStil, Texte deutsch/englisch, Speichern
    optionen.py        Optionen je Plot: Standardwerte, Prüfung, Aufruf
    profile.py         Plots je Betriebspunkt
    flaechen.py        3D-Flächen
    heatmaps.py        Kennzahlen-Übersicht
    paritaet.py        Paritätsdiagramme
tests/                 Tests und Referenzergebnisse
docs/                  Handbuch (Markdown), Bilder, PDF-Erzeugung
```

Abhängigkeiten: Die Skripte im Projektordner lesen `config.py` und rufen das Paket `taylorflow` auf. Das Paket selbst kennt `config.py` nicht; alle Einstellungen werden als Argumente übergeben. Die Funktionen lassen sich also auch einzeln nutzen.

12.2 Tests

```
python -m pytest
```

Datei	Prüft
`tests/test_regression.py`	wertet alle mitgelieferten Messdateien aus und vergleicht jede Zahl der Betriebspunkt-Tabellen und einen Auszug der Profile exakt mit `tests/referenz/`
`tests/test_bausteine.py`	Einzelteile mit synthetischen Daten: Dateinamen, Fit, aktive Zone, kLa_log_mean, Unsicherheit, Modellregistry, Potenzansatz
`tests/test_dateiauswahl.py`	Auswahl der Messdateien (Liste, „alle“, Auswahlfenster), Dateien der Videoauswertung
`tests/test_plots.py`	Plot-Optionen in `config.py` und `abbildung.py` für jeden Plot

Nach jeder Code-Änderung zeigt der Regressionstest, ob sich Ergebnisse verändert haben.

Wenn eine Änderung die Ergebnisse absichtlich verändert (neues Modell, Korrektur):

1. Die Wirkung beziffern:

   `python tests/referenz_erneuern.py`

   listet jede abweichende Spalte mit der größten relativen Abweichung.

2. Die Referenz neu schreiben:

   `python tests/referenz_erneuern.py --schreiben`

3. `__version__` in `taylorflow/__init__.py` erhöhen; die Version steht in jeder `<Kanal>_meta.json`.

12.3 Typische Erweiterungen

12.3.1 Neues Literaturmodell

In `taylorflow/literatur.py`:

```
def mein_modell(u_B, L_B, L_S, d_h, D):
    """Autor (Jahr), Zeitschrift. Gültig für …"""
    return 2.0 * np.sqrt(D * u_B / (L_B + L_S)) / d_h

LITERATURMODELLE["mein_modell"] = Literaturmodell(
    "mein_modell", "Autor 2027", "kLa_lit_mein_modell_1_s",
    lambda x: mein_modell(*_e(x, "u_B", "L_B", "L_S", "d_h", "D")))
```

Die verfügbaren Eingangsgrößen stehen in `modelleingaben()`. Neue Größen werden dort und beim Aufruf in `pipeline.py` und `modellvergleich.vorhersage` ergänzt. Die neue Spalte ändert die Betriebspunkt-Tabelle; danach die Referenz neu schreiben (12.2).

12.3.2 Neuer Plot

1. Funktion in einem Modul unter `taylorflow/plots/` anlegen. Sie bekommt die Daten, einen `PlotStil`, einen Titel und ihre Optionen als Schlüsselwort-Argumente und gibt eine Figure zurück. Für den Aufbau:

   - `neue_figur(stil, groesse_cm)`
   - `t(schluessel, stil)` für Beschriftungen; neue Texte in `TEXTE` eintragen, jeweils deutsch und englisch

- `formatiere_achsen(ax, stil)`

2. In `taylorflow/plots/optionen.py`:
   - in `STANDARD` einen Eintrag mit allen Optionen und Standardwerten
   - in `erzeuge` einen Zweig, der die Funktion aufruft und den Dateinamen festlegt
3. In `config.PLOTS` einen Block mit denselben Einträgen wie in `STANDARD` anlegen. Der Test `test_config_zeigt_alle_optionen_mit_standardwerten` prüft, dass beide übereinstimmen.

Danach steht der Plot in `auswertung.py` und `abbildung.py` zur Verfügung.

12.3.3 Neue Option für einen vorhandenen Plot

Parameter mit Standardwert in der Plotfunktion ergänzen, denselben Namen mit demselben Standardwert in `STANDARD` (`optionen.py`) und im Block in `config.PLOTS` eintragen.

12.3.4 Andere Flüssigkeit oder anderes Gas

- **Flüssigkeit:** `fluessigkeit` in `config.PARAMETER` setzen (Name für thermo, z. B. `"ethanol"`).
- **Henry-Konstante:** Für andere Flüssigkeiten als Wasser `stoffdaten.henry` erweitern; derzeit gibt es eine Fehlermeldung.
- **Diffusionskoeffizient:** `stoffdaten.diffusion_co2_wasser` gilt nur für CO2/Wasser. Er wird nur für die Literaturmodelle und die Spalte `D_CO2_m2_s` verwendet.
- **Gas:** Neue Gase in `HENRY_DATEN` eintragen.

12.3.5 Neuer Auswerte-Parameter

1. Feld mit Standardwert in `AuswerteParameter` (`parameter.py`) anlegen.
2. In der Rechnung über `p.<name>` verwenden.
3. In `config.PARAMETER` aufführen und in Kapitel 5 dokumentieren.

12.4 Handbuch erzeugen

Das Handbuch besteht aus den Markdown-Dateien in `docs/` (englisch in `docs/en/`). Bilder und PDF werden mit zwei Skripten erzeugt:

```
python docs/bilder_erzeugen.py          Bilder aus den mitgelieferten Messdaten
 ↪  (deutsch)
python docs/bilder_erzeugen.py en       dasselbe mit englischer Beschriftung
python docs/build_pdf.py                docs/Handbuch_TaylorFlow.pdf
python docs/build_pdf.py en             docs/Handbuch_TaylorFlow_en.pdf
```

Für das PDF werden zusätzlich `pip install pypandoc_binary` und eine LaTeX-Installation mit XeLaTeX (TeX Live oder MiKTeX) benötigt.